

Fields as intermediates, functionals as outputs

A rigorous account of Phase 0 of the surrogate programme

Michele Barucca — Institute of Mathematics, EPFL

17 September 2026

Abstract

A steady magnetohydrodynamic solution for one aluminium reduction cell costs about seven hours. A trained surrogate returns one in milliseconds — but it returns a *velocity field*, and a velocity field is not a decision. What an engineer acts on is a number computed from it: a kinetic energy, a peak interface elevation, a measure of how much current runs horizontally through the metal pad.

The obvious route is to regress those numbers on the anode currents directly. It fails, frequently worse than predicting their mean. This note explains why, and shows that the remedy is to predict the field and evaluate the number on it exactly. Whether that helps, and by how much, is governed by a single property of the number being computed, which we state as a rule and verify on thirty of them: some gain nothing from a field model, some are recovered by a crude one, and some require an accurate one.

Applied to the diagnostic that originally motivated a *nonlinear* surrogate, the rule reverses that conclusion — the underlying field is affine in the anode currents to 99.8% of its variance, and the nonlinearity belongs to the definition of the diagnostic rather than to the cell. Three further results bound what the existing surrogate can be trusted to do: its advantage over a linear map is interpolation rather than extrapolation, its sensitivity operator is rank-deficient on the interface but not on the velocity, and that rank deficiency predicts, in advance, which inverse design problems have reproducible solutions. Everything is measured on an existing campaign of 1594 simulations, with no new solver runs.

Contents

1	The project, and what this note adds	2
1.1	What already exists	2
1.2	What this note asks	3
2	Functionals of a predicted field	5
2.1	The question	5
2.2	Why regressing the observable fails	6
2.3	What the field route costs, and what decides it	7
2.4	Results	9
2.5	Where the advantage actually lives	9
2.6	Tier B: the five diagnostic planes, and the role of the functional's class	10
2.7	Tier C: the horizontal current in the metal	12
3	The linearity boundary	13
3.1	The same question, asked of the functionals	15
4	The sensitivity operator	16
4.1	What the network adds, measured as a subspace	18

5	Well-posedness of the inverse problem	19
6	How many simulations the map needs	22
7	Two proposed improvements, both refuted	23
7.1	Retraining at the predictability truncation	23
7.2	Imposing volume conservation on the interface	24
8	Items established for the geometry extension	24
8.1	Is the transfer construction needed at all?	25
8.2	Applying the relabelling to solved runs	26
8.3	Gate 1: the rank of the geometry direction	26
8.4	Incompressibility across geometries	30
8.5	Does the transfer reduce the rank? A free pilot says no	32
8.6	Corrections to the transfer construction	33
8.7	Orthogonal designs for estimating A	34
8.8	What the campaign costs, and what fits	35
9	Summary of what is established and what is not	36
10	The question this opens	39
11	The feeding question, pursued	41
11.1	The parameterisation was already present	42
11.2	What the objective costs to measure	42
11.3	Two results withdrawn	42
11.4	The mesh	43
11.5	The coarse mesh is the usable instrument	43
11.6	What this blocks	44
A	How every quantity in this note is measured	44
B	Reproduction	46

1 The project, and what this note adds

1.1 What already exists

The object under study is a surrogate for the steady flow in a CNG2-type reduction cell. It is trained on a campaign of 1 594 converged magnetohydrodynamic solutions, all of the same cell at the same geometry, differing only in how the total line current of 490 kA is distributed over the 24 anodes. Every run is in the constant-ACD regime, so the anode underside is displaced with the interface and the gap is held fixed pointwise; this matters later, because it is what keeps the leading force term linear in the currents. Each run stores the converged velocity field, the interface elevation, and some fifty derived scalars computed by the solver itself.

The campaign was not sampled at random. Each current vector was drawn from one of seven named patterns, and the counts are uneven by design:

regime	runs	how the current vector was built
gaussian	1 198	smooth random perturbation about uniform
gradient	100	a monotone ramp along the cell
cluster	80	a contiguous group of anodes raised or lowered
weak	76	one anode driven far below the box
single	70	one anode at a box limit, the rest rebalanced
dead	69	one anode carrying essentially no current
uniform	1	every anode equal; the reference run

The mix is deliberate and reflects how a cell is actually run. **gaussian** is three quarters of the campaign because smooth departures from uniform are the generic operating condition. The remaining regimes are rare but real: a **dead** or **weak** anode is what a cell looks like around an anode replacement, a scheduled operation every anode undergoes on a cycle of weeks, while **cluster** and **single** cover localised disturbances. They are in the campaign because they happen, not as stress tests — which is what makes Section 3 a question about coverage rather than about discarding awkward data — and what makes Section 2.5 report every result by operating condition rather than pooled.

Chapter 2 of the thesis built three surrogates from this campaign, one per target field, each a proper orthogonal decomposition followed by a residual feed-forward network mapping the 24 currents to the modal coefficients. The three targets, used unchanged here, are

name	field	entries M	modes k	reported R^2
midacd	velocity on the mid-gap plane	18 483	64	0.966
full3d	velocity on the fluid nodes	602 775	64	0.977
interface	interface elevation	11 025	48	0.927

midacd is the fibre-averaged velocity on the plane halfway up the anode–cathode gap, the plane the solver already exports for its own diagnostics; **full3d** is the same velocity on the metal and bath nodes only, excluding anodes and cathode blocks; **interface** is the elevation of the metal–bath interface on its own surface mesh.

1.2 What this note asks

Chapter 2 establishes that the fields are predicted well. It does not establish that the *quantities computed from them* are, and those are what a cell operator reads. Nor does it separate the surrogate’s accuracy inside the training distribution from its accuracy outside. This note answers both, together with two questions that follow from them — how the response depends on each anode, and whether that dependence can be inverted — using only the runs that already exist.

Throughout, the input is written $\vec{p}$: the per-anode current expressed as a fractional deviation from the mean,

$$p_j = \frac{I_j}{I_{\text{mean}}} - 1, \quad I_{\text{mean}} = \frac{I_{\text{tot}}}{24} = \frac{490\,000}{24} \text{ A}.$$

The potline delivers a fixed total current and every ampere passes through some anode, so $\sum_j I_j = I_{\text{tot}}$ is not a property of the data but a constraint on it. In these coordinates that constraint becomes an identity:

$$\sum_{j=1}^{24} p_j = \frac{1}{I_{\text{mean}}} \sum_{j=1}^{24} I_j - 24 = \frac{I_{\text{tot}}}{I_{\text{tot}}/24} - 24 = 0. \quad (1)$$

Raising one anode necessarily lowers others; there is no operating point in which the deviations do not cancel.

This is also why the model is given $\vec{p}$ rather than the currents themselves. Across the campaign the total is 490 000 A to within 0.06 A, so $I_j = I_{\text{mean}}(1 + p_j)$ with the same I_{mean} in every run: $\vec{p}$ is the identical information with a constant removed. Supplied raw, every input would read 1.00 ± 0.07 — a unit carrying nothing, plus the seven percent carrying everything. Subtracting the one also places the uniform-current reference at the origin, which is what the Δ -learning of Definition 1 needs, and turns an affine constraint into a linear subspace, without which neither the null-space arguments of Section 4 nor the orthogonal design of Section 8.7 would apply directly.

One caveat, small but load-bearing later. The solver returns currents that meet the constraint to seven digits rather than exactly, so $|\sum_j p_j| \leq 1.6 \times 10^{-6}$ over the campaign rather than 0. The direction $\vec{1}$ is therefore very nearly, but not exactly, unexercised, and the smallest singular value of the input design is 1.5×10^{-6} relative rather than zero. Section 4 truncates above that value; a threshold below it would leave the operator unidentified while appearing conservative. The per-anode limits $16\,400 \leq I_j \leq 24\,400$ A become $p_j \in [-0.197, 0.195]$, so the feasible set is a compact convex polytope of affine dimension 23, not 24.

Equation (1) is worth dwelling on, because four later results are consequences of it and of nothing else: the sensitivity operator of Section 4 is rank-deficient by exactly one; the direction $\vec{1}$ is never exercised by any run, so the data cannot constrain the response along it; an optimiser must be parameterised on the constraint hyperplane rather than projected onto it afterwards (Section 5); and the orthogonal design of Section 8.7 satisfies the constraint automatically, because its rows sum to zero by construction.

A discretised field is written $g_h(\vec{p}) \in \mathbb{R}^M$. Four indices recur and each is reserved:

index	runs over	count
j	the anodes	24
m	the retained modes	k (64, 64, 48)
i	the nodes of a discretised field	M
n	the runs of the campaign	$N = 1\,594$

In particular k is only ever a *count* of modes, never an index; the anode index is j throughout.

How every number in this note is measured — which runs, which split, which quadrature, and which of two R^2 conventions — is set out in Appendix A; it is worth reading before disputing a figure, and can be skipped otherwise.

Definition 1 (Reduced-order surrogate). Let $\vec{u}_{\text{ref}} := g_h(\vec{I}_{\text{ref}})$ be the run at uniform current (stat_0001), let $\mathbf{G} \in \mathbb{R}^{N \times M}$ have rows $g_h(\vec{I}_i) - \vec{u}_{\text{ref}}$, let $\bar{g}_h := \frac{1}{N} \sum_i (g_h(\vec{I}_i) - \vec{u}_{\text{ref}})$ and $\mathbf{G}_c := \mathbf{G} - \vec{1} \bar{g}_h^\top$. With the truncated singular value decomposition, taken by the method of snapshots [1, 2], $\mathbf{G}_c \approx \mathbf{U}_k \mathbf{\Sigma}_k \mathbf{V}_k^\top$, $\mathbf{V}_k^\top \mathbf{V}_k = \mathbf{I}_k$, the surrogate is

$$g_h(\theta; \vec{I}) = \vec{u}_{\text{ref}} + \bar{g}_h + \mathbf{V}_k \vec{c}(\theta; \vec{I}), \quad (2)$$

with $\vec{c}(\theta; \cdot) : \mathbb{R}^{24} \rightarrow \mathbb{R}^k$ a residual feed-forward network trained by minimising $\mathcal{L}(\theta) = \frac{1}{Nk} \sum_n \sum_m (c_m(\theta; \vec{I}_n) - c_m(\vec{I}_n))^2$. The split is stratified by campaign regime into 1 194/237/163 runs; the basis $\mathbf{V}_k$, the mean $\bar{g}_h$ and the target statistics are computed on the training rows only.

Provenance Every number in this note is reproducible from the scripts named in Appendix B, against the campaign master `master_ml.h5` ($N = 1\,594$ accepted runs), with `/home/barucca/minico`

2 Functionals of a predicted field

2.1 The question

The quantities an engineer reads are not fields but *functionals* of fields: a kinetic energy, a root-mean-square speed, a peak elevation. Write such a quantity as $\Phi(g_h)$ for a functional Φ .

Chapter 2’s headline scores — 0.977, 0.966, 0.927 — are scores on the *fields*. They say how much of the run-to-run variation of a 602 775-entry array the surrogate reproduces, and they say nothing whatever about $\Phi(g_h)$. A model can capture 88% of a field and under half of a functional of it, and in Section 2 one does. Every R^2 from here to the end of this section is therefore a score on a *single number*, computed with equation (10) on the 163 test runs, and is not comparable with the field scores quoted above; Appendix A keeps the two conventions apart.

Two routes lead from the currents to such a number. Regress it on the currents directly, or predict the field and evaluate Φ on it exactly. The first is the obvious one and it fails badly — Section 2.2 measures the failure and explains it, and the numbers there are the worst in this note by design, since they are the thing to be improved on. Section 2.3 gives the mechanism by which the second route recovers them, and Section 2.4 the results: on the same functional where direct regression scores -0.258 , evaluating Φ on the network’s field scores 0.971.

Definition 2 (Four predictors). Fix a functional Φ and a train/test split. Let $\mathbf{D}_{\text{lin}}$ be the design matrix of the 24 normalised currents and $\mathbf{D}_{\text{quad}}$ that of the $\binom{24}{2} + 24 = 300$ products $p_i p_j$. The predictors are

$$(i) \quad \hat{\Phi}_{\text{lin}} = \text{least squares of } \Phi \text{ on } [\vec{\mathbf{I}} \ \mathbf{D}_{\text{lin}}]; \quad (3)$$

$$(ii) \quad \hat{\Phi}_{\text{quad}} = \text{screened hierarchical ridge on } [\vec{\mathbf{I}} \ \mathbf{D}_{\text{lin}} \ \mathbf{D}_{\text{quad}}]; \quad (4)$$

$$(iii) \quad \hat{\Phi}_{\text{fld}} = \Phi(\vec{u}_{\text{ref}} + \bar{g}_h + \mathbf{V}_k \mathbf{A} \vec{p}); \quad (5)$$

$$(iv) \quad \hat{\Phi}_{\text{net}} = \Phi(g_h(\theta^*; \vec{\mathbf{I}})). \quad (6)$$

Predictors (i) and (ii) *regress the observable*; (iii) and (iv) *regress the field and then evaluate the observable exactly*. In (ii) the ridge penalty is applied to the quadratic block alone, so that $\alpha \rightarrow \infty$ recovers (i) and the family is nested; the products are screened against the residual of (i), retaining the 40 largest, and both the screened and the full design are offered with α chosen on a validation slice carved out of the training rows.

Definition 3 (The truth and the floor). Every predictor is scored against one reference, *the truth*:

$$\Phi_{\text{true}} = \Phi(g_h),$$

the functional of Definition 4 applied to the solver’s field at full resolution — every node, every component, nothing approximated.

One further quantity is reported beside it, *the floor*:

$$\Phi_{\text{floor}} = \Phi(\vec{u}_{\text{ref}} + \bar{g}_h + \mathbf{V}_k \mathbf{V}_k^T (g_h - \vec{u}_{\text{ref}} - \bar{g}_h)),$$

the same functional of the same true field after a round trip through the k retained modes. No model enters it: the coefficients are obtained by projecting the true field, not by predicting it. The floor is therefore what a predictor of the form (5)–(6) would score with *zero* regression error, and it splits such a predictor’s residual into the part a better regression could remove and the part only a larger basis could. A floor at 1.000 says truncation costs nothing and the whole residual is regression.

Definition 4 (Quadrature and functional class). Let $\vec{w} \in \mathbb{R}^{M_{\text{node}}}$, $\sum_i w_i = 1$, be the lumped $\mathbb{P}_1$ mass weights: for tetrahedra, w accumulates $|\det(\cdot)|/6/4$ per vertex; for surface triangles, $\text{area}/3$ per vertex. Weights are taken once from the reference run, so that a single fixed functional is applied to every run. Functionals are classified by the norm they are continuous in:

$$\underbrace{\Phi(g) = \sum_i w_i |g_i|}_{L^1}, \quad \underbrace{\Phi(g) = \left(\sum_i w_i |g_i|^2\right)^{1/2}}_{L^2}, \quad \underbrace{\Phi(g) = \max_i g_i}_{L^\infty}. \quad (7)$$

2.2 Why regressing the observable fails

Proposition 5 (Expansion of a quadratic functional). *Let $\Phi(g) = \|g\|_{\vec{w}}^2 := \sum_i w_i |g_i|^2$ and write $g_h = \vec{u}_{\text{ref}} + \Delta\vec{u}$. Then*

$$\Phi(g_h) = \underbrace{\|\vec{u}_{\text{ref}}\|_{\vec{w}}^2}_{\text{constant}} + \underbrace{2\langle \vec{u}_{\text{ref}}, \Delta\vec{u} \rangle_{\vec{w}}}_{\text{linear in } \Delta\vec{u}} + \underbrace{\|\Delta\vec{u}\|_{\vec{w}}^2}_{\text{quadratic in } \Delta\vec{u}}. \quad (8)$$

If moreover $\Delta\vec{u}$ is an affine function of $\vec{p}$, the three terms are respectively constant, affine and quadratic in $\vec{p}$, so Φ is a quadratic polynomial in $\vec{p}$ and is in the span of predictor (ii).

Proof. Bilinearity of $\langle \cdot, \cdot \rangle_{\vec{w}}$; composition of a quadratic form with an affine map is a quadratic polynomial. $\square$

Proposition 5 shows that Φ retains a linear part, so a linear regression of Φ on $\vec{p}$ should not fail outright. The reason it does is the relative size of the two varying terms.

How much of Φ any linear model can reach Of the three terms above, the first is the same in every run and only the second is linear, so a linear regression of Φ on the currents can only reach what that second term carries. It carries little, because the perturbation $\Delta\vec{u}$ is close to perpendicular to the field it perturbs, and a perpendicular perturbation contributes nothing to an inner product with $\vec{u}_{\text{ref}}$.

Rather than infer the ceiling from that expansion, it can be measured directly: fit the best affine function of the 24 currents to Φ and see how much of its variation is left.

the single number being predicted	from the field	mean $ \cos \theta $	R^2 in sample	R^2 held out
u_{rms}^2 , mean square speed	full3d	0.064	0.242	−0.258
u_{rms}^2 , mean square speed	midacd	0.050	0.221	−0.156
η_{std}^2 , elevation variance	interface	0.248	0.673	0.533

Each row is *one scalar per run* — 1 594 numbers, not 1 594 fields. The middle column names the field it is computed from, and is there only to say which mesh the weights come from.

Every column, and how to recompute it. Write $\langle \vec{a}, \vec{b} \rangle_{\vec{w}} := \sum_i w_i a_i b_i$ and $\|\vec{a}\|_{\vec{w}} := \langle \vec{a}, \vec{a} \rangle_{\vec{w}}^{1/2}$ for the weighted inner product and norm, with $\vec{w}$ the weights of Definition 4.

mean $|\cos \theta|$ the cosine of the angle between the perturbation and the field it perturbs,

$$\cos \theta_n = \frac{\langle \Delta\vec{u}_n, \vec{u}_{\text{ref}} \rangle_{\vec{w}}}{\|\Delta\vec{u}_n\|_{\vec{w}} \|\vec{u}_{\text{ref}}\|_{\vec{w}}}, \quad \Delta\vec{u}_n = g_h(\vec{p}_n) - \vec{u}_{\text{ref}}, \quad (9)$$

reported as $\frac{1}{N} \sum_n |\cos \theta_n|$ over all $N = 1\,594$ runs. It lies in $[0, 1]$; zero is perpendicular, one is parallel.

R^2 in sample fit $\hat{\Phi}(\vec{p}) = b_0 + \sum_j b_j p_j$ by ordinary least squares on the 1 194 training runs, then score it on those same runs. This is the most any linear model could extract if generalisation were free.

R^2 **held out** the *same* coefficients b , scored on the 163 test runs.

Both R^2 columns use the same formula,

$$R^2 = 1 - \frac{\sum_n (\Phi_n - \hat{\Phi}(\vec{p}_n))^2}{\sum_n (\Phi_n - \bar{\Phi})^2}, \quad (10)$$

with $\bar{\Phi}$ the mean over whichever set is being scored; they differ only in that set. A negative value means the fit is worse on unseen runs than simply predicting their average. The last column is not an estimate of predictor (i): it *is* predictor (i), and it reproduces the corresponding entries of Table 1 to four decimals (-0.2582 and -0.1560 against the -0.258 and -0.156 reported there). Both are computed on the solver's own field, at full resolution, with the weights of Definition 4; η_{std}^2 is a variance about each run's own weighted mean, so for that row the expansion is applied to the centred elevation. (Script `cross.py`.)

A linear model fitted to Φ explains under a quarter of its variation even on the runs it was fitted to, and generalises worse than the mean. That is the behaviour of column (i) throughout Table 1: $R^2 \in [-0.29, 0.58]$, negative on all six velocity functionals. Nothing is wrong with the fit; there is almost nothing there to fit.

Why the interface is the exception The interface row behaves completely differently — a linear model reaches 0.533 held out, where the velocities are negative — and that is why predictor (i) scores 0.578 on η_{std} in Table 1 while failing on every velocity functional. The reason is in the first column: the currents move the interface in a direction five times closer to the shape it already has ($|\cos| = 0.248$ against 0.050–0.064), so the linear term of the expansion survives. This is a statement about how the currents act on the interface, not about the interface as an object.

Where the orthogonality comes from The anode currents *reorganise* the flow rather than rescale it. This is a property of the constant-ACD regime, in which the anode underside is displaced with the interface, $H_{\text{an}}^{k+1} = H^{k+1} + \text{ACD}$, so that the gap is preserved pointwise and the leading force term $\vec{j}^{\text{hor}} \times \vec{B}^{\text{hor}}$ is linear in $\vec{I}$.

2.3 What the field route costs, and what decides it

Predictor (5) or (6) does not fit Φ at all: it fits $\Delta\vec{u}$ and then applies Φ exactly. It does *not* inherit the field model's accuracy, and what follows says what it does inherit.

Proposition 6 (Error amplification under a quadratic functional). *Let $\Delta\vec{u} = \widehat{\Delta\vec{u}} + \vec{e}$ with $\vec{e}$ the prediction error, and let $\Phi = \|\cdot\|_{\vec{w}}^2$. Then*

$$\Phi(\vec{u}_{\text{ref}} + \widehat{\Delta\vec{u}}) - \Phi(\vec{u}_{\text{ref}} + \Delta\vec{u}) = -2\langle \vec{u}_{\text{ref}} + \widehat{\Delta\vec{u}}, \vec{e} \rangle_{\vec{w}} - \|\vec{e}\|_{\vec{w}}^2. \quad (11)$$

In particular the error in Φ contains a term $-\|\vec{e}\|_{\vec{w}}^2$ which is systematically signed and does not average out, and a cross term which is $O(\|\vec{u}_{\text{ref}}\| \|\vec{e}\|)$.

Proof. Expand both sides by bilinearity and cancel. $\square$

The residual is what decides it, not the model Equation (11) contains $\vec{e}$ and nothing else about the field model. Whether that model is linear, quadratic or a network does not appear; only the size of what it leaves behind does, and it appears twice, once linearly through the cross term and once quadratically through a bias that does not average away.

Write $\rho := \text{Var}(\vec{e}) / \text{Var}(\Delta\vec{u})$ for the fraction of the field the model fails to explain. This note measures three cases, and they are the whole argument:

field	functional	class	(i) lin	(ii) quad	(iii) lin-fld	(iv) net-fld
midacd	u_{mean}	L^1	-0.164	0.905	0.545	0.993
	u_{rms}	L^2	-0.162	0.926	0.572	0.985
	ke	L^2	-0.156	0.944	0.488	0.982
	u_{max}	L^∞	0.042	0.913	0.099	0.667
full3d	u_{mean}	L^1	-0.288	0.880	0.028	0.974
	u_{rms}	L^2	-0.263	0.904	0.256	0.974
	ke	L^2	-0.258	0.804	0.149	0.971
	u_{max}	L^∞	-0.133	-0.867	0.182	0.918
interface	η_{std}	L^2	0.578	0.718	0.692	0.945
	η_{max}	L^∞	0.147	0.510	0.131	0.954
	η_{min}	L^∞	-0.529	-7.478	0.725	0.837
	η_{ptp}	L^∞	0.204	-0.690	0.635	0.909
	η_{mean}	L^1	0.008	-0.679	0.058	0.887

Table 1: Held-out R^2 of the four predictors of Definition 2 against the truth, on the 163-run test split, with mass-weighted functionals. Best in each row in bold. Script `field_functionals.py`.

field	functional	(iv) net-fld	floor	floor - (iv)
midacd	u_{mean}	0.993	0.999	0.006
	u_{rms}	0.985	1.000	0.015
	ke	0.982	1.000	0.018
	u_{max}	0.667	0.967	0.300
full3d	u_{rms}	0.974	1.000	0.026
	u_{max}	0.918	0.990	0.072
interface	η_{std}	0.945	1.000	0.055
	η_{max}	0.954	0.998	0.044
	η_{mean}	0.887	0.876	-0.011

Table 2: The truncation floor and the residual gap. Because the floor is ≈ 1 on every L^1/L^2 functional, the entire error of predictor (iv) there is *regression* error and none of it is basis error: a better network would improve these, a larger k would not. The exception is η_{mean} , where the floor itself is 0.876 and (iv) has already attained it — consistent with η_{mean} being the residual of the volume constraint rather than a physical signal.

field model	field it predicts	ρ	R^2 of Φ built on it
linear map on the currents	velocity	0.128	0.03–0.73
trained network	velocity	0.023	0.84–0.99
linear map on the currents	current density	0.0018	1.000

The first two rows invite the conclusion that a network is needed. The third refutes it: the *same* linear field model, on a field it happens to fit to 99.82%, delivers a functional score of 1.000 and beats the network on the same quantity (Section 2.7). What separates success from failure is ρ , and the model class matters only in so far as it sets ρ . For velocity a linear map leaves too much and a network does not; for the current density a linear map already leaves almost nothing.

The rule to carry away is therefore not “use a network” but *evaluate the functional on the most accurate field model available, whatever its form* — and ρ , not R^2 of the functional, is the number to look at when deciding whether the field route will work on a new quantity.

2.4 Results

The functional agrees with the solver’s own ALUCCELL computes several of these quantities itself, to its own definitions, and comparing against them checks that Φ reads the same field the solver does. Over the 1 525 bulk runs: for every L^∞ functional the agreement is *exact*, with Pearson correlation 1.000000 and ratio 1.0000. The remaining differences are fully accounted for by the definitions. η_{std} has ratio 0.9214, the area weighting of Definition 4 against the solver’s unweighted node mean. For u_{mean} and u_{rms} on **full3d** the ratios are 2.586 and 1.657, and these factor exactly into the two definitional differences at once:

	measured ratio	domain	$\times$ weighting
u_{mean}	2.586	2.000	1.293
u_{rms}	1.657	$1.414 = \sqrt{2}$	1.172

The first factor is the domain: the solver divides a fluid-only sum by all 401 838 mesh nodes where this note divides by the 200 925 fluid ones, and $401\,838/200\,925 = 2.000$ — the velocity is *exactly* zero at all 200 913 non-fluid nodes, verified, so the extra nodes contribute nothing but denominator, and the $\sqrt{\cdot}$ in the r.m.s. turns the factor into $\sqrt{2}$. The second is the quadrature: Definition 4 weights by nodal volume while the solver takes an unweighted node mean, and the mesh is graded, with the largest elements in the metal pad where the flow is fastest, so volume weighting raises the mean by 29%. All correlations lie in $[0.9949, 1.000000]$.

An earlier version of this measurement attributed the whole of 2.586 to the node counts. That accounts for 2.000 of it and not the rest; both factors are needed, and both are now verified run by run.

2.5 Where the advantage actually lives

The scores of Table 1 are pooled over the whole test split, and three quarters of that split is **gaussian**. That hides which operating conditions the result rests on, and the answer changes the reading. Taking ke on **full3d** as representative:

condition of the cell	runs	(i) lin	(ii) quad	(iv) net-fld
gaussian , ordinary operation	121	−2.68	0.967	0.994
gradient , a ramp along the cell	10	−3.95	0.992	0.998
cluster , a group disturbed	8	−0.69	0.979	0.993
single , one anode at a limit	8	−0.20	0.986	0.982
weak , an anode being replaced	8	−5.19	−0.475	0.784
dead , an anode disconnected	8	−71.5	0.737	0.884

In ordinary operation a screened quadratic regression of the scalar is almost as good as evaluating it on a predicted field — 0.967 against 0.994 — and a reader could reasonably conclude that the field route is a refinement. On a cell with an anode being replaced it is *negative*, while the field route still returns 0.784. The same reversal appears on every target: 0.106 against 0.943 for u_{mean} on **midacd**, and −15.3 against 0.694 for η_{min} on the interface.

So the result is not that field-then-functional is better on average It is that direct regression fails exactly where a model is most wanted. An operator does not consult a surrogate to learn what a cell does in ordinary operation; they consult it when something is unusual, and the unusual conditions here are not contrived — **weak** and **dead** are what a cell looks like around an anode replacement, which every anode undergoes on a cycle of weeks, and **single** is a localised disturbance. Those eight-run groups carry the argument, and the 121 **gaussian** runs that dominate the pooled number do not.

This is the same conclusion Section 3 reaches from the other direction and on a different quantity. There the comparison is linear against network on the *field*, with the whole campaign in training, and the network’s margin is widest on the same conditions — 2.4% against 11.8% during an anode replacement. Here it is direct regression against the field route on a *functional*. Two independent measurements, the same operating conditions deciding both.

One condition nothing predicts The exception is worth stating rather than smoothing. For $u_{\max}$ on **weak**, predictor (iv) returns 0.031 and predictor (ii) -20.5 : the peak velocity during an anode replacement is not recovered by anything measured here. A maximum is an extremum of a field whose largest departure from the reference is precisely in that regime, so both the truncation and the regression are at their worst simultaneously. Any use of this surrogate for a short-circuit or peak-velocity margin under a replaced anode is unsupported by these measurements.

A caveat on η_{mean} Its scores are poor everywhere, including -5.1 for the best predictor on **gaussian**. This is an artefact of the observable and not of any model: volume conservation holds the area-weighted mean interface displacement at essentially zero (Section 7.2), so its between-run variance is near zero and R^2 , which divides by that variance, is not a meaningful score for it.

2.6 Tier B: the five diagnostic planes, and the role of the functional’s class

The plane integrals of `ml_export.cpp` were reimplemented in vectorised NUMPY (`plane_integrals.py`): for every tetrahedron cut by the plane the intersection polygon is formed, fanned into triangles and integrated with the same quadrature the solver uses,

$$\text{ke} = \sum_T A_T (|v_0|^2 + |v_1|^2 + |v_2|^2 + v_0 \cdot v_1 + v_0 \cdot v_2 + v_1 \cdot v_2) / 12, \quad (12)$$

$$\text{net flux} = \sum_T A_T (v_{01}^a + v_{12}^a + v_{02}^a) / 3, \quad \text{half abs flux} = \sum_T A_T (|v_{01}^a| + |v_{12}^a| + |v_{02}^a|) / 6, \quad (13)$$

with v_{ij}^a the mean of the slice-axis velocity component at the two endpoints. The five planes are the solver’s own diagnostic planes — one horizontal at the interface datum and four vertical cross-sections — and the mesh geometry is held at the reference run, as for the mass weights of Definition 4. Their coordinates are in Appendix A.

Validation of the reimplementation On the reference run, where the fixed geometry is the run’s own, all fifteen quantities reproduce the solver to six significant figures for ke and half abs flux. On other runs the two agree to within 0.5%, the residue being the frozen geometry. The exception is net flux, which is a near-cancelling quantity — of order 0.2% of the absolute flux, since $\int \vec{u} \cdot \vec{n} = 0$ exactly for an incompressible flow in a closed cell — and whose value is therefore dominated by the mesh rather than by the field. Its rows below should be read as a diagnostic of the discretisation, which is what Chapter 2 uses it for, and not as a physical observable.

Proposition 7 (For an affine functional, predictors (i) and (iii) are the same estimator). *Let Φ be affine. Then $\hat{\Phi}_{\text{fld}} = \Phi(\vec{u}_{\text{ref}} + \bar{g}_h + \mathbf{V}_k \mathbf{A} \vec{p})$ is an affine function of $\vec{p}$, hence lies in the hypothesis class of predictor (i). Since (i) is the least-squares optimum within that class, $R_{\text{train}}^2(\text{i}) \geq R_{\text{train}}^2(\text{iii})$, and field-then-functional can offer no advantage over direct regression for such a functional.*

Proof. Composition of an affine map $\vec{p} \mapsto \vec{u}$ with an affine Φ is affine; the least-squares fit minimises the training residual over all affine predictors. $\square$

plane	quantity	(i) lin	(ii) quad	(iii) lin-fld	(iv) net-fld	floor
<i>ke — quadratic (L^2)</i>						
mid-ACD	ke	−0.299	0.916	0.450	0.988	0.998
+X	ke	0.833	0.609	0.756	0.959	0.991
−X	ke	0.799	0.920	0.912	0.987	0.994
+Y	ke	0.189	0.858	0.617	0.992	0.997
−Y	ke	0.174	0.703	−0.160	0.900	0.996
<i>half absolute flux — absolute (L^1)</i>						
mid-ACD	haf	−0.088	0.760	0.979	0.991	0.985
+X	haf	0.859	0.869	0.730	0.942	0.993
−X	haf	0.806	0.904	0.856	0.980	0.994
+Y	haf	0.217	0.767	0.945	0.994	0.999
−Y	haf	−0.151	−0.323	0.984	0.998	0.999
<i>net flux — affine, and a discretisation diagnostic</i>						
mid-ACD	nf	0.264	0.812	0.262	0.788	0.929
+X	nf	0.864	0.477	0.864	0.901	0.937
−X	nf	0.880	0.860	0.881	0.922	0.836
+Y	nf	0.871	0.958	0.870	0.969	0.979
−Y	nf	0.419	0.481	0.428	0.923	0.871

Table 3: The fifteen plane functionals, held-out R^2 against the truth on the 163-run test split. Predictor (iv) is best on fourteen of fifteen rows. Note the third block: for the affine functional, (i) and (iii) agree to within 0.009 throughout.

The prediction is confirmed to three decimals The five net-flux rows of Table 3 give (i, iii) = (0.264, 0.262), (0.864, 0.864), (0.880, 0.881), (0.871, 0.870), (0.419, 0.428): identical to within 0.009, the residue being the POD truncation and out-of-sample scatter. No other block of Table 3 behaves this way.

Corollary 8 (A classification of functionals). *Tables 1, 3 and 4 together support a three-way classification, which is the transferable content of this section.*

1. **Affine** Φ (net flux). Predictor (iii) coincides with predictor (i) by Proposition 7; nothing is to be gained, and direct regression is already correct.
2. **Absolute, L^1 -type** Φ (half absolute flux, Φ_{frac}). The absolute value destroys affineness in the observable but not in the field, so predictor (iii) already recovers it: 0.979, 0.945, 0.984 against −0.088, 0.217, −0.151 for predictor (i). A linear field model suffices.
3. **Quadratic, L^2 -type** Φ (kinetic energy, r.m.s. speed). Proposition 6 applies and the field model’s residual is squared, so predictor (iii) is only partially successful (−0.160 to 0.912) and the network is required: predictor (iv) reaches 0.900–0.992.

The rule is therefore not “always predict the field” but: an affine observable needs no field; an absolute observable needs a field of any kind; a quadratic observable needs an accurate one.

Where the affine observables come from The pattern of predictor (i) across Table 3 also explains an observation of Chapter 2. The $\pm X$ planes score 0.80–0.88 under a linear regression for *every* quantity including the quadratic ones, while the mid-ACD and $\pm Y$ planes score near zero or below. The $\pm X$ planes cut the cell near its ends, where the base flow is large, so in the expansion (8) the cross term $2\langle \vec{u}_{\text{ref}}, \Delta \vec{u} \rangle$ is correspondingly large and the quantity inherits the base flow’s linearity. This is exactly the mechanism Chapter 2 proposes for why “the quantities with a large value already at uniform current” are the linear ones, now measured on five planes rather than asserted.

functional	class	(i) lin	(ii) quad	(iii) lin-fld	(iv) net-fld	floor
Φ_{frac}	L^1	-0.107	0.912	1.000	0.982	1.000
Φ_{max}	L^∞	-0.257	0.458	0.998	0.984	1.000

Table 4: Held-out R^2 against the truth for the metal current functionals (14), on the same 163-run split as Table 1. Here predictor (iii), the *linear* field model, is the best of the four.

2.7 Tier C: the horizontal current in the metal

The sharpest single result of Chapter 2 concerns the horizontal current fraction in the metal pad, the standard proxy for metal-pad instability, defined in the aggregation code as an unweighted element-sum ratio over the 644 160 elements with `cueb_refs = ref_alu`:

$$\Phi_{\text{frac}} = \frac{\sum_e \|\vec{j}_e^{xy}\|}{\sum_e |j_e^z|}, \quad \Phi_{\text{max}} = \max_e \|\vec{j}_e^{xy}\|. \quad (14)$$

Chapter 2 reports $R_{\text{lin}}^2 = 0.024 \rightarrow R_{\text{quad}}^2 = 0.952$ for Φ_{frac} regressed directly on the currents and concludes: “*this is the quantitative evidence that a nonlinear surrogate is warranted.*” The field `fields_full/courant` needed to test that conclusion exists only in the full campaign master, so the experiment was run on the cluster; only the result was returned.

The current density in the metal is almost exactly linear in the currents A proper orthogonal decomposition of the metal current density over the same 1 194 training runs gives an effective rank of 5.9 — throughout this note that means the participation ratio $R_{\text{eff}} = (\sum_i \pi_i^2)^{-1}$ with $\pi_i = \sigma_i^2 / \sum_j \sigma_j^2$, which counts how many directions carry comparable energy and is not an integer — with 48 modes retaining 99.997% of the variance. The *linear* map from the 24 currents to those 48 modal coefficients attains $R^2 = 0.9982$, against 0.9659 for the trained network on the same coefficients. The current density is the most linear object measured anywhere in this campaign.

These two figures score a *field* — they are pooled over all 48 coefficients of all 163 test runs, the second convention of Appendix A — and must not be read against the numbers in Table 4 below, each of which scores a *single scalar per run*. The whole point of what follows is that the two can disagree violently: 0.9982 on the field and -0.107 on a number computed from it.

Proposition 9 (The nonlinearity is manufactured by the observable). *For Φ_{frac} the underlying current-density field is reproduced by an affine map of the currents to 99.82% of its variance (Section 2.7), yet the scalar Φ_{frac} is not reproduced at all by an affine map of the same currents ($R^2 = -0.107$ on the 163 test runs). The two statements are consistent because Φ_{frac} is a ratio of sums of Euclidean norms: neither $\vec{j} \mapsto \sum_e \|\vec{j}_e^{xy}\|$ nor a quotient is an affine functional, so an affine $\vec{p} \mapsto \vec{j}$ composed with them is not affine in $\vec{p}$. Evaluating the same non-affine functional on the affinely predicted field recovers it exactly, $R^2 = 1.000$.*

Consequence for Chapter 2’s argument The inference “the scalar is not linear in $\vec{I}$, therefore a nonlinear surrogate is warranted” does not hold for this quantity, and this is the quantity on which Chapter 2 rests that argument. What the measurement supports instead is the opposite: for Φ_{frac} a *linear* surrogate of the field, followed by exact evaluation, is not merely adequate but optimal to three decimal places, and it is the *network* that is slightly worse (0.982) because the affine map is already essentially exact and the network adds estimation variance. The physics is consistent with this: given the anode currents and a fixed conductivity field, $\vec{j} = \sigma \nabla V$ with V solving a linear elliptic problem, and in the constant-ACD regime the conductivity field barely moves — the same mechanism by which Chapter 2 finds the cell resistance *pinned* ($R^2 = -0.013$ at a coefficient of variation of 1.5%).

A unit-scale trap in the pipeline, found here The first attempt at predictor (iv) on this field failed, and the cause was not the architecture but the target scale. The deployed configurations disable input normalisation on the grounds that POD coefficients are already scaled by the decomposition. That is true of velocity coefficients and false of current-density coefficients, which are eight orders of magnitude larger; standardising the targets recovers the scores reported above. Any new target field carrying different physical units requires this to be revisited. The setting is calibrated to one field and is silent, not safe, for others.

What Tier C adds to Table 1 Tier C completes the picture that the velocity results only half showed. Predictor (iii) fails on velocity because the linear field model leaves 12% of the variance unexplained and Proposition 6 amplifies it; it succeeds perfectly on the current because the linear field model leaves 0.18%. The governing quantity is therefore the *field* model’s residual, exactly as (11) states, and not whether the model is linear or a network. The correct general statement is: *evaluate the functional on the most accurate field model available*, whatever its functional form.

The L^∞ exception, stated rather than smoothed One row of Table 1 goes the other way: for `midacd  $u_{\max}$` , predictor (ii) scores 0.913 against 0.667 for predictor (iv). A maximum is not continuous with respect to the L^2 norm in which the POD is optimal, and the mid-ACD field is additionally a *fibre-averaged* quantity carrying fine spanwise banding. The same functional on `full13d`, which has no banding, reverses the ordering (0.918 against -0.867). The honest conclusion is that the functional’s continuity class, and not only the field’s accuracy, decides whether (6) is preferable; L^1 and L^2 functionals are safe, L^∞ functionals must be checked individually.

What this settles Two objections to the surrogate programme are answered by Table 1. First, that the surrogate predicts quantities which are not decisions: one trained network supplies *every* functional in the table at 0.84–0.99, including functionals never fitted for, whereas each direct regression must be fitted per functional, requires that functional to have been pre-computed on all N runs, and returns $R^2 < 0$ on four of thirteen. Second, that a linear model would suffice: it does not, for the functionals — predictor (iii) is a linear field model and it loses to (ii) almost everywhere. Section 3 qualifies this second answer sharply.

3 The linearity boundary

The surrogate beats a linear map comfortably, and this section asks what that margin is made of. The answer is that it is made of coverage.

With the campaign as sampled, the network is better than a linear map in *every* regime, by factors of two to fourteen, and the margin is largest exactly where the cell is least ordinary:

regime of the test run	linear	network	factor
gradient	0.029	0.002	13.7
gaussian	0.047	0.005	8.8
single	0.053	0.015	3.6
cluster	0.070	0.014	4.9
dead (an anode replacement)	0.118	0.024	4.9
weak	0.202	0.104	1.9

Relative L^2 error of the reconstructed `full13d` field on the test split; lower is better. During an anode replacement the network is wrong by 2.4% where a linear map is wrong by 11.8%, and

	linear	quadratic	network
<i>full campaign in training</i> (1 194/163, from Chapter 2)			
midacd	0.8817	0.9190	0.9660
full13d	0.8715	0.9125	0.9771
interface	0.7162	0.7483	0.9266
<i>normal envelope only in training</i> (973/427, Definition 10)			
midacd	0.5808	−4.2064	0.5854
full13d	0.5754	−4.1399	0.2023
interface	0.2892	−1.2293	0.2955

Table 5: Between-run R^2 of the modal coefficients, pooled over the test set. The two blocks are scored on *different* test populations and are not comparable row by row; what is comparable is the *margin* between predictors within a block. Script `compare_baselines.py`.

that is the moment an operator would most want a model, because the cell is not in its usual state.

Train instead on the smooth regimes alone and hold the rest out, and the margin collapses to a fraction of a percent and on one target reverses. But so does everything else: no predictor of any class beats the mean on a condition it has never seen. The conclusion is therefore about sampling and not about model class. A surrogate interpolates; asking it to reach an anode replacement it was never shown is asking the wrong thing of it. This bears on two later decisions — what a geometry campaign must cover (Section 6), and whether the surrogate can be trusted at an optimiser’s solution, which tends to sit at a corner of the design space (Section 5).

Definition 10 (Regime-restricted split). The campaign generator labels each run with a regime $\in \{\textit{gaussian}, \textit{gradient}, \textit{cluster}, \textit{single}, \textit{weak}, \textit{dead}, \textit{uniform}\}$. In the restricted split the training and validation rows are drawn only from *gaussian* and *gradient* — the normal operating envelope — and *every* run of every other regime is placed in the test set. Because the basis is fitted with `fit_idx = train_idx`, this restricts $\mathbf{V}_k$, $\bar{g}_h$ and the target statistics to the normal envelope as well. The resulting partition is 973/194/427, of which 296 test runs are held-out regimes and 131 are in-distribution.

The aggregate scores of the lower block are not self-explanatory, because the test set of Definition 10 is dominated by a few regimes.

The held-out regimes carry almost all the variance Of the total test sum of squares of the modal coefficients on `full13d`, *weak* carries 49.6% and *dead* 37.7%; the five held-out regimes together carry 96.7%, while the in-distribution *gaussian* and *gradient* runs carry 3.2% and 0.1%. The pooled R^2 of the lower block of Table 5 is therefore, to within a few percent, a measurement on *weak* and *dead* alone.

The network’s margin is interpolation, not extrapolation With the full campaign in training, the network exceeds the linear map by 0.084 (`midacd`), 0.106 (`full13d`) and 0.210 (`interface`). With the rare regimes removed from training and used as the test set, the margin collapses to +0.0046 (`midacd`) and +0.0063 (`interface`), and *inverts* to −0.373 on `full13d`.

Proposition 11 (Interpretation). *Tables 5 and 6 are consistent with a single statement: greater model flexibility improves interpolation and degrades extrapolation. Both refer to the restricted split, where only the smooth regimes are in training. There the network beats the linear map by factors of 3.7 and 8.8 on the two regimes it was trained on, and is worse by 1.26 and 1.28 on dead and weak, which it has never seen. With the whole campaign in training the network is*

field	predictor	in distribution		held out			
		gaussian	gradient	cluster	single	dead	weak
full13d	linear	0.01398	0.01113	0.05609	0.03372	0.30911	0.32979
	quadratic	0.00729	0.00284	0.04845	0.02703	1.05618	1.01551
	network	0.00375	0.00127	0.02698	0.01950	0.38943	0.42248
midacd	linear	0.01470	0.01186	0.05763	0.03477	0.34304	0.34534
	quadratic	0.00824	0.00338	0.05060	0.02964	1.14342	1.06752
	network	0.01660	0.01351	0.05833	0.03524	0.34042	0.34386

Table 6: Relative L^2 error on the reconstructed field, by campaign regime, for models fitted on the normal envelope only. The network is uniformly the best predictor *inside* the envelope and on the milder held-out regimes, and it is the worst of the three on *dead* and *weak* for full13d. Together with Section 3 this accounts for the inverted aggregate.

better on all six (Section 3); the two statements differ in what was trained on, not in what was measured. Because those two regimes carry 87.3% of the test variance (Section 3), the pooled score inverts. The margin reported in Chapter 2 is therefore obtained because the rare regimes are in the training set, and it measures interpolation between neighbouring current vectors of the same regime.

Corollary 12 (Campaign-design consequence). *A surrogate that must be accurate during an anode replacement requires anode-replacement runs in its training set. It cannot reach them by extrapolation from the smooth regimes, and a more flexible model makes this worse rather than better. The campaign analysed here already meets that requirement; the corollary is addressed to the geometry campaign of Section 8, which must meet it again in a larger design space. The relative L^2 error of the linear map rises from 0.014 on in-distribution gaussian runs to 0.330 on weak — a factor of 24 — and the network’s rises by a factor of 113. A campaign that spends 79% of its budget on gaussian runs, as the present one does, is mis-allocated for that purpose; the orthogonal box-saturating design of Proposition 22, which places every anode at a bound in every run, is the corrective.*

The polynomial is not merely worse, it is unusable Predictor (ii) attains $R^2 = -4.21$, -4.14 and -1.23 in the restricted setting, that is, worse than the constant predictor. With 324 predictors fitted on 973 rows and evaluated outside the convex hull of the training design, a quadratic polynomial extrapolates catastrophically: its relative L^2 error reaches 1.07 on *weak* and 1.14 on *dead* (midacd), i.e. an error as large as the signal. Note that inside the envelope the same polynomial is the second-best predictor (Table 6). A polynomial control is appropriate within the design and must not be used outside it.

Optimisation-level evidence The restricted networks reached their best validation score at epochs 56, 58 and (full13d) within a 320 s run, with early stopping triggered on all three, against epochs 496–498 and no early stopping on the full campaign. Less data and a narrower input distribution both reduce the epoch at which generalisation stops improving.

3.1 The same question, asked of the functionals

Section 2 compares the four predictors on functionals using the whole campaign; this section compares two of them on *fields* using the normal envelope. The experiment that has not been run is the intersection, and it is the one an operator would care about: with the rare conditions removed from training, is a linear model enough for the numbers actually read?

the number predicted	from	(i) lin	(ii) quad	(iii) lin-flt	(iv) net-flt	floor
ke	full13d	−0.731	0.283	−7.647	−15.918	0.977
u_{rms}	full13d	−0.795	0.109	−5.124	−8.811	0.979
ke	midacd	−0.665	0.461	−5.533	−5.292	0.990
u_{rms}	midacd	−0.712	0.228	−3.508	−3.354	0.991
η_{std}	interface	0.275	−1.498	0.589	0.589	1.000
η_{ptp}	interface	0.121	−1.366	0.638	0.634	0.974

Table 7: Held-out R^2 of the functionals when only the normal envelope is in training, on the 427-run test set of Definition 10. Compare Table 1, where the whole campaign is in training and predictor (iv) scores 0.84–0.99.

Nothing works on the velocity, and the field route works worst Every predictor of a velocity functional is worse than predicting its mean, except direct quadratic regression — the predictor Section 2 spends its length arguing against. The field route does not merely lose its advantage, it *inverts*: −15.9 against −0.73 for plain linear regression of the same quantity.

This is the amplification of Proposition 6 running backwards. The floor stays at 0.98, so the basis represents these runs perfectly well and truncation is not the problem; what fails is the regression, which reaches a coefficient R^2 of only 0.20 on this split (Table 5). A residual that large, squared by a quadratic functional, destroys it. The rule of Section 2.3 predicted exactly this: the field route is worth taking only when the field residual is small, and here it is not.

And linear equals the network, again Predictors (iii) and (iv) agree to three decimals on every row. Whatever the network contributes over a linear map, it contributes only where it has seen data — the conclusion of Table 5, now confirmed on the quantities an engineer reads rather than on the fields.

The limitation this places on Section 2 The headline result of this note — that evaluating a functional on a predicted field beats regressing it directly — holds *with the rare regimes in training*. It does not survive their removal. Since three quarters of the campaign is **gaussian** and the rare regimes carry almost all of the held-out variance, the two statements are not in conflict, but the second bounds the first: the field route is a statement about interpolation, like the network’s own advantage, and not a way of extrapolating beyond the sampled regimes. The practical reading is not that the field route should be avoided but that the campaign must contain the conditions it will be asked about — which this one does, by design. On the interface, where the residual stays small even on unseen regimes, the gain survives regardless.

4 The sensitivity operator

Two questions follow from having a surrogate at all. How much does each anode matter — not to a single scalar, but to the whole flow? And can that be run backwards, to choose currents that produce a wanted flow?

Both are answered by one object: the matrix of derivatives of the field with respect to the currents. This section builds it, shows that its spectrum can be computed on a small matrix rather than a large one, and establishes that it is rank-deficient by exactly one — a consequence of the fixed total current, not of the data. Section 5 then shows that the *shape* of that spectrum predicts, before any optimisation is run, which inverse problems have reproducible answers and which do not. The velocity fields and the interface behave quite

field	σ_1	σ_{23}/σ_1	eff. rank of $\mathbf{A}$	dirs. below $5\%\sigma_1$
midacd	1.44	0.175	12.0/23	0
full13d	12.0	0.242	13.1/23	0
interface	1.56	0.013	5.6/23	5

Table 8: Spectrum of $\mathbf{A}$; effective rank as defined in Section 2.7. The velocity fields are well conditioned; the interface is not. That the interface is the delicate object is consistent with the aluminium literature, which discusses its stability on physical grounds [22, 23], though the deficiency measured here is a property of the estimated map and not of the Sele criterion.

differently, and the difference matters because the quantities an operator would actually want to control are interface quantities.

Definition 13 (Sensitivity operator). Let $\mathbf{A} \in \mathbb{R}^{k \times 24}$ be the least-squares solution of $\vec{c} \approx \mathbf{A} \vec{p}$ on the training rows, and let

$$\mathbf{S} := \mathbf{V}_k \mathbf{A} \in \mathbb{R}^{M \times 24}. \quad (15)$$

Column j of $\mathbf{S}$ is the field $\partial g_h / \partial p_j$: the response of the whole cell to a deviation of anode j . $\mathbf{S}$ is the Jacobian of the field with respect to the currents at the uniform-current operating point, estimated from snapshots rather than by differentiating the solver.

The object is not new and the name used here is loose. In inverse problems and parameter estimation the same matrix is the *sensitivity matrix*, and the reading of its singular spectrum — which directions are resolved, which are annihilated, and where to truncate — is the subject of [4, 5]. In design optimisation it is what an adjoint computes one column at a time and at the cost of one solve per objective [8, 9]; the snapshot route taken here gives the whole operator instead, at the cost of a designed campaign. And $\mathbf{A}^\top \mathbf{A}$ is exactly the active-subspace matrix $\mathbb{E}[\nabla f \nabla f^\top]$ of [6] for an affine f , which is what Section 4.1 exploits to extend the diagnostic to the network.

Lemma 14 (The spectrum is computable on the small matrix). $\mathbf{S}^\top \mathbf{S} = \mathbf{A}^\top \mathbf{A}$, hence $\mathbf{S}$ and $\mathbf{A}$ have the same singular values.

Proof. $\mathbf{S}^\top \mathbf{S} = \mathbf{A}^\top \mathbf{V}_k^\top \mathbf{V}_k \mathbf{A} = \mathbf{A}^\top \mathbf{A}$ by orthonormality of the columns of $\mathbf{V}_k$. $\square$

So the conditioning of a map into a space of six hundred thousand dimensions is read from the singular values of a matrix with 24 columns. This is what makes every spectrum in this section cheap enough to compute for all three targets.

Proposition 15 (Rank deficiency and the minimum-norm representative). *Because $\sum_j p_j = 0$ for every run, the training design satisfies $\mathbf{D}_{\text{lin}} \vec{1} = \vec{0}$, so $\vec{1} \in \ker \mathbf{D}_{\text{lin}}$ and $\text{rank } \mathbf{A} \leq 23$. Consequently $\mathbf{A}$ is not identified: for any $\beta \in \mathbb{R}^k$ the matrix $\mathbf{A} + \beta \vec{1}^\top$ gives identical predictions on the feasible set. Every prediction-based diagnostic is blind to this; the columns of $\mathbf{A}$ are not. Truncating singular values below $\mathbf{rcond} = 10^{-4}$ selects the minimum-norm representative, in which the rows of $\mathbf{A}$ sum to zero and the coefficients are read as deviations, consistently with the inputs.*

The threshold is not arbitrary There is a wide window of admissible truncation thresholds, and the value used here sits inside it while an apparently safer, smaller value would sit below the null direction and truncate nothing at all — silently leaving the operator unidentified. The bounding ratios are in Appendix A.

Three ranks that must not be conflated The campaign reports three distinct participation ratios and they answer different questions:

1. the *design* effective rank, 19.0/23, a property of the sampled current vectors (PCA of the $N \times 24$ input matrix);
2. the *response* effective rank, 13.1/13.9/5.8, a property of the output snapshots (POD of the field);
3. the effective rank of $\mathbf{A}$, Table 8, a property of the *map*.

Only the third governs the inverse problem. A design of full rank can produce a response of low rank, and a map may annihilate input directions that the design populates perfectly.

Response low-rankness is inherited from the map If the design were isotropic on the constraint hyperplane, the response covariance would be exactly $\mathbf{A}\mathbf{A}^\top$ and the two spectra would coincide. The design populates 19 of 23 directions, hence is close to isotropic, and the effective ranks agree closely: 12.0 against 13.1, 13.1 against 13.9, 5.6 against 5.8. The compressibility of the flow is therefore largely a statement that *the forcing is low-rank*, which is a more transferable claim than a statement about the fluid.

Relation to active subspaces For a general $f(\vec{p})$ the active subspace is spanned by the dominant eigenvectors of $\mathbf{C} = \mathbb{E}[\nabla f \nabla f^\top]$. For an affine f , $\nabla f \equiv \mathbf{A}^\top$ and $\mathbf{C} = \mathbf{A}^\top \mathbf{A}$, so the active subspace is exactly the span of the leading right singular vectors of $\mathbf{A}$. The construction therefore extends Definition 13 to the trained network by automatic differentiation, and comparing the two subspaces measures whether the network has found input directions the linear map did not.

4.1 What the network adds, measured as a subspace

Section 2.7 showed a case in which the network is *worse* than the affine map, which makes the question “what does the network actually contribute” concrete rather than rhetorical. The active subspace answers it directly.

The construction is the active subspace of [6, 7], applied to a trained network rather than to a simulator. It asks a different question from variance-based global sensitivity analysis [13]: not how much variance each input explains, but which input *directions* move the output.

Network active subspace Let $\mathbf{C}_{\text{net}} := \mathbb{E}[\mathbf{J}^\top \mathbf{J}]$ with $\mathbf{J} = \partial \vec{c}(\theta^*; \vec{p}) / \partial \vec{p} \in \mathbb{R}^{k \times 24}$ obtained by automatic differentiation, the expectation taken over the 163 test runs. Write $\mathbf{C}_{\text{net}} = \mathbf{W}_{\text{net}} \mathbf{\Lambda} \mathbf{W}_{\text{net}}^\top$ and $s_i := \sqrt{\lambda_i}$. For an affine model $\mathbf{J} \equiv \mathbf{A}$, so $\mathbf{C} = \mathbf{A}^\top \mathbf{A}$ and $s_i = \sigma_i$: the two constructions are directly comparable. Subspace agreement at rank r is measured by $\text{ov}(r) = \|\mathbf{W}_{\text{lin},r}^\top \mathbf{W}_{\text{net},r}\|_F^2 / r \in [0, 1]$, equal to 1 iff the two r -dimensional subspaces coincide.

On the velocity the network adds gain, not direction For full13d the leading subspaces largely coincide, $\text{ov}(1) = 0.868$ and $\text{ov}(3) = 0.884$, while the network’s gradients are uniformly *larger* along them, by factors 1.29 to 1.47 on the first six directions and only 1.02–1.05 on directions 22 and 23. The network has therefore found essentially the input directions the affine map already identifies, and has added curvature along them rather than new directions.

i	full3d			interface		
	$\sigma_i(\mathbf{A})$	$s_i(\text{net})$	ratio	$\sigma_i(\mathbf{A})$	$s_i(\text{net})$	ratio
1	11.9997	16.1804	1.348	1.5597	1.3584	0.871
2	9.0826	12.3426	1.359	1.1232	1.2362	1.101
4	6.8279	10.0417	1.471	0.8096	0.9515	1.175
22	3.1967	3.2700	1.023	0.0296	0.0726	2.457
23	2.9012	3.0505	1.051	0.0197	0.0615	3.130
24	0	0.3783	—	0	0.0462	—
<i>subspace overlap</i> $\text{ov}(r)$						
$r = 1$		0.868			0.452	
$r = 3$		0.884			0.662	
$r = 5$		0.845			0.806	
$r = 23$		0.998			0.957	

Table 9: Active subspace of the trained network against the linear sensitivity operator. Script `active_subspace.py`.

On the interface it does find different directions For `interface` the leading direction differs substantially, $\text{ov}(1) = 0.452$, a principal angle of 47.8° , and the network places 2.5 to 3.1 times more gradient than the affine map on directions 22 and 23 — precisely the directions the operator’s spectrum calls nearly null ($\sigma_{23}/\sigma_1 = 0.013$, Table 8). This is consistent with the interface being the target on which the quadratic terms buy the most ($R_{\text{lin}}^2 = 0.404 \rightarrow R_{\text{quad}}^2 = 0.703$), and it is also consistent with the interface inverse problem being the one whose network landscape is most rugged (Section 5) (Table 10).

A spurious sensitivity to an infeasible direction Both networks have a *non-zero* gradient along $\vec{1}$: $s_{24} = 0.378$ and 0.046 , against $\sigma_{24}(\mathbf{A}) = 0$ exactly. By Proposition 15 that direction leaves the feasible polytope and is never exercised by any run, so nothing in the data or in the loss constrains it, and the network is free to place whatever sensitivity there it likes; the least-squares fit, by contrast, annihilates it by construction through the singular-value truncation. The consequence is harmless for prediction, since no admissible $\vec{p}$ has a component along $\vec{1}$, but it is not harmless for *design*: a gradient-based optimiser that does not parameterise on the constraint hyperplane will follow it. This is a concrete argument for optimising over $\vec{p} = \mathbf{B}\vec{y}$ with $\mathbf{B}$ an orthonormal basis of $\ker(\vec{1}^\top)$, rather than projecting a 24-dimensional gradient after the fact, and for adding $\|\mathbf{J}\vec{1}\|$ to the diagnostics of any future architecture comparison.

5 Well-posedness of the inverse problem

A millisecond surrogate invites the inverse question: which currents produce a wanted flow? Whether that question has a well-defined answer is decided before any optimisation is run, by the conditioning of the operator of Section 4.

This section tests that claim independently, and finds something sharper than expected. Optimising from many random starting points either converges to one answer or does not, and which of the two happens is predicted, in both ordering and magnitude, by a spectrum computed from a small matrix. But the failure, when it occurs, belongs to the *surrogate* and not to the cell: replace the network by the affine field model and every problem here becomes convex, with one answer reached from every start.

field	Φ	σ_{23}/σ_1	obj. spread	soln. spread	verdict
full13d	ke	0.242	0.001%	0.3%	well posed
midacd	ke	0.175	7.17%	10.7%	multi-modal
interface	η_{std}	0.013	5.30%	36.7%	multi-modal

Table 10: Multi-start reproducibility of (16), with the two spreads and the verdict as defined above. Constraints are satisfied to $\leq 1.2 \times 10^{-10}$ A and every optimum saturates the per-anode box, as an almost-affine response over a polytope requires. Script `inverse_design_probe.py`.

Design problem Given a functional Φ , solve

$$\min_{\vec{p}} \Phi(g_h(\theta^*; \vec{p})) \quad \text{subject to} \quad \sum_{j=1}^{24} p_j = 0, \quad p_j \in [-0.19673, 0.19510], \quad (16)$$

that is, over the feasible polytope of affine dimension 23. Gradients are taken by automatic differentiation through the network and the reconstruction (2), so the optimiser sees the exact surrogate derivative; (16) is solved by SLSQP from 20 independent uniform random feasible starts. Each start r returns a feasible point, in amperes $\vec{I}_r$, and an achieved objective Φ_r ; the table reports two summaries of the twenty, and a verdict that is a rule rather than a judgement.

objective spread the relative range of what was achieved,

$$\frac{\max_r \Phi_r - \min_r \Phi_r}{|\min_r \Phi_r|}.$$

Small means the starts agree on the *value* even if they disagree on how to reach it.

solution spread how far apart the optima are, as a fraction of how far an anode could possibly move. Over all $\binom{20}{2} = 190$ pairs,

$$\text{median}_{r < s} \frac{\|\vec{I}_r - \vec{I}_s\|_2}{\sqrt{24} (I_{\max} - I_{\min})}.$$

The $\sqrt{24}$ converts a norm over 24 anodes into a root-mean-square *per* anode, in amperes, and the box width 8000 A makes it dimensionless. So 36.7% means two starts typically land some 2900 A per anode apart — more than a third of the entire admissible range.

verdict solution spread below 2% gives *well posed*; otherwise an objective spread below 1% gives *null space*, the same value reached by different currents; otherwise *multi-modal*.

The last distinction is what the experiment is for. A null space is benign: it means the objective does not care, and a regulariser may pick among the optima freely. Genuinely distinct optima are not benign, because they have genuinely different values, and no regulariser removes them.

Proposition 16 (With an affine field model the problem is convex). *Let $\vec{u}(\vec{p}) = \vec{u}_0 + \mathbf{V}\mathbf{A}^\top \vec{p}$ and $\Phi(\vec{u}) = \vec{u}^\top \mathbf{W} \vec{u}$ with $\mathbf{W}$ the positive quadrature weights. Writing $\vec{c} = \mathbf{A}^\top \vec{p}$,*

$$\Phi = k_0 + 2\vec{b}^\top \vec{c} + \vec{c}^\top \mathbf{M} \vec{c}, \quad \mathbf{M} = \mathbf{V}^\top \mathbf{W} \mathbf{V}.$$

$\mathbf{M}$ is symmetric positive definite, because $\mathbf{W}$ has positive entries and $\mathbf{V}$ orthonormal columns, so Φ is strictly convex in $\vec{c}$. By Proposition 15, $\text{rank } \mathbf{A} = 23$, which is the dimension of the constraint hyperplane, so $\vec{p} \mapsto \vec{c}$ is injective there and Φ is strictly convex on the feasible set. A strictly convex function on a compact convex set has a unique minimiser.

The statement covers every quadratic objective used here, and also the *field-matching* problem $\min_{\vec{p}} \|\vec{u}(\vec{p}) - \vec{u}^*\|_{\vec{w}}^2$ — “which currents produce this flow pattern” — which is of the same form. An optimiser needs a scalar objective, but that scalar may perfectly well be a distance to a wanted field; nothing here is restricted to quantities like a kinetic energy.

The multi-modality is the network’s, not the cell’s Running the identical probe on the affine field model confirms Proposition 16 and settles what the spreads mean:

Φ on	affine field model		network	
	obj. spread	soln. spread	obj. spread	soln. spread
<code>full13d</code>	0.0000%	0.01%	0.001%	0.30%
<code>midacd</code>	0.0000%	0.05%	7.17%	10.70%
<code>interface</code>	0.0000%	0.00%	5.30%	36.70%

The affine model reaches the same optimum from every start, to solver tolerance, on all three. **The distinct optima of Table 10 are therefore created by the network’s nonlinearity and are not a property of the cell.** Script `inverse_affine_probe.py`.

This is the same mechanism as the spurious sensitivity of Section 4.1, where both networks were found to place gradient along the one direction the constraint makes infeasible and no run exercises. Where the data does not constrain the map, the network is free, and that freedom shows up downstream as structure that is not in the physics — here, as local minima. It also explains why the conditioning predicts the spread at all: σ_{23}/σ_1 measures how many directions are weakly constrained, and the interface has five below five percent of σ_1 .

What follows for inverse design Not that the network is useless, but that it should not be the thing optimised. The affine model is convex and has a unique answer; the network is the more accurate map. The construction that uses both is to optimise the affine model inside a trust region and let the network choose where that region sits — which is what the next phase proposes for an unrelated reason, namely that the network has no adjoint and no gradient consistency with the solver, and which this measurement now independently supports.

Conditioning predicts reproducibility The ordering of the solution spread in Table 10 is the ordering of σ_{23}/σ_1 in Table 8, and the magnitudes track: the best-conditioned map (`full13d`, 0.242) yields an essentially unique optimum (25 A per anode across all starts), while the worst-conditioned (`interface`, 0.013, with five directions below 5% of σ_1) yields optima differing by 2933 A per anode. A spectrum computed from a 64×24 matrix in seconds therefore predicts the behaviour of a non-convex optimisation over a 23-dimensional polytope.

Three failure modes, distinguished It is worth separating cases that are often conflated. If the solution spread is small, the problem is *well posed*. If the objective spread is below 1% while the solution spread is large, there is a *null space*: the optimiser drifts along directions the objective cannot see, and the remedy is Tikhonov regularisation with λ on the L-curve, or restriction to the leading right singular vectors of $\mathbf{A}$. If *both* spreads are large, the landscape is *multi-modal*: the optima are genuinely distinct local minima with genuinely different objective values, and no regularisation removes them — multi-start becomes mandatory and the spread, not a single optimum, is the result. Table 10 shows the third case for the two objectives of practical interest, and Proposition 16 then says where those minima come from: not from the cell, whose affine model is convex, but from the surrogate being asked to play the part of the objective.

The optimum lies where the surrogate is weakest Every optimum in Table 10 saturates the box constraint, which is expected: a near-affine response minimised over a polytope attains its extremum at a vertex. But the campaign places 79% of its runs in the *gaussian* regime, near the centre of $\mathcal{P}$, and the per-regime error of Chapter 2 rises from 0.0054 there to 0.0244 at a dead-anode corner and 0.1045 in the *weak* corner. Combined with Corollary 12, the surrogate error at a design optimum should be assumed to be of order 10% rather than the aggregate 0.5%, and no claimed improvement smaller than that is meaningful without solver verification.

n	full13d		midacd		interface
	linear	quadratic	linear	quadratic	linear
50	0.589	-3.29 ± 4.78	0.597	-9.56 ± 13.9	0.235
100	0.720	0.00 ± 1.01	0.719	0.13 ± 0.83	0.509
200	0.759	-0.24 ± 0.78	0.774	-0.23 ± 0.76	0.480
400	0.812	-0.04 ± 0.91	0.826	0.08 ± 0.91	0.563
600	0.855	0.888 ± 0.024	0.867	0.899 ± 0.020	0.635
800	0.865	0.864 ± 0.044	0.876	0.819 ± 0.121	0.695
1000	0.860	0.899 ± 0.016	0.871	0.907 ± 0.014	0.677
1194	0.872	0.916	0.882	0.920	0.716

Table 11: Learning curves: held-out R^2 of the modal coefficients against the number of training runs. This is the *field* convention of Appendix A — one pooled ratio over all coefficients of all 163 test runs — and is not comparable with the scalar scores of Table 1. Mean over three stratified draws, with the standard deviation shown for the quadratic predictor, whose variability is the point. Script `learning_curves.py`.

6 How many simulations the map needs

The campaign cost thousands of core-hours and the geometry extension of Section 8 would cost more, so it is worth knowing how many runs the surrogate actually needed. Refitting it on subsamples, with the basis held fixed, answers that.

The two model classes behave qualitatively differently. The linear map saturates smoothly and early; the quadratic map has a threshold below which it is not merely worse but unusable. The existing campaign is well past both, which is the quantitative basis for the budget of the next phase. The same experiment exposes something less comfortable: adding rare-regime runs to the training set makes the normal envelope worse, not better.

Learning curve For $n \in \{50, 100, 200, 400, 600, 800, 1000, 1194\}$ draw a subsample of the training rows *stratified by regime*, so that the regime proportions of the full campaign are preserved and the rare regimes are not lost at small budget; fit predictors (i) and (ii) of Definition 2 on the subsample, targeting the modal coefficients; score pooled on the fixed 163-run test split. Three independent draws per n . The basis $\mathbf{V}_k$ is held at the one fitted on all 1194 training rows, so the curve answers “given the basis, how many runs are needed to learn the map”; the cost of refitting the basis is not included.

Saturation The linear predictor reaches 94% of its final score at $n = 400$ and 99% at $n = 800$ on both velocity fields; the increment from 800 to 1194 runs is 0.007 and 0.006, within the draw-to-draw scatter. The interface saturates more slowly and is still rising at $n = 1194$.

The quadratic predictor has a threshold, not a curve Below $n \approx 600$ the quadratic predictor is not merely inaccurate but unstable: at $n = 50$ it scores -3.3 ± 4.8 and -9.6 ± 13.9 , and the standard deviation across draws exceeds the mean in magnitude up to $n = 400$. At $n \geq 600$ it becomes both accurate and reproducible (0.888 ± 0.024 , 0.899 ± 0.020). This is the behaviour predicted by counting: the screened design of Definition 2 carries $24 + 40 = 64$ coefficients, so fivefold oversampling requires $n \approx 320$ and tenfold $n \approx 640$.

Budget For the linear operator $\mathbf{A}$ at a fixed geometry, $n \approx 400$ –600 runs suffice; the present campaign of 1194 is roughly 2.5 times larger than the linear map requires, and about 2 times larger than the screened quadratic map requires. This is the quantitative basis for the multi-geometry budget of Section 8.7: a sweep geometry does not need a full campaign.

field	k	total	truncation	regression	<i>weak</i> total
interface	48	0.00155	0.00025	0.00153	0.01491
	7	0.00448	0.00430	0.00100	0.02269
midacd	64	0.02287	0.01414	0.01753	0.15391
	24	0.04369	0.04168	0.01100	0.17491
full3d	64	0.01947	0.01485	0.01185	0.13307
	24	0.04288	0.04174	0.00766	0.16436

Table 12: Relative L^2 error of the retrained surrogates against the *full-resolution* field, $\|\hat{\vec{u}}_k - \vec{u}^{\text{true}}\|_2 / \|\vec{u}^{\text{true}}\|_2$ averaged over the 163 test runs, decomposed as in (17). The last column repeats the total on the *weak* runs alone, the regime with the largest perturbation. Lower is better; bold marks the deployed truncation, which wins every comparison.

Adding rare-regime runs degrades the normal envelope The per-regime columns of the learning curve show a trade-off that a pooled score conceals. As n grows the linear map improves sharply on the high-variance regimes — *dead* rises from 0.33 to 0.93 on *midacd* — and *degrades* on the bulk: *gaussian* falls from 0.850 to 0.815 on *midacd* and from 0.750 to 0.434 on the interface, and *single* falls from 0.727 to 0.253 on the interface. A single global map must compromise between populations of very different amplitude, and the variance-weighted objective resolves the compromise in favour of the large-amplitude regimes. Together with Proposition 11 this says that neither a normal-envelope model nor a global model is uniformly best, and that regime-aware or locally weighted fitting is the natural next question.

7 Two proposed improvements, both refuted

Chapter 2 proposes two changes described as cheap and obviously beneficial. Both were implemented and measured, and neither is beneficial. They are recorded because the reasons are instructive and because the first is a measurement artefact of the existing tooling.

7.1 Retraining at the predictability truncation

The proposal is to retrain at $k \approx 24/24/7$ rather than 64/64/48, on the grounds that modes beyond the predictability limit carry variance the currents do not determine and can only add noise. The first half of that is true; the conclusion does not follow.

Error decomposition Let $\vec{u}^{\text{true}}$ be the field at full resolution, Π_k the L^2 projection onto the span of $\mathbf{V}_k$ about the reference, and $\hat{\vec{u}}_k$ the surrogate prediction at truncation k . Write

$$\underbrace{\|\hat{\vec{u}}_k - \vec{u}^{\text{true}}\|}_{\text{total}} \leq \underbrace{\|\Pi_k \vec{u}^{\text{true}} - \vec{u}^{\text{true}}\|}_{\text{truncation}} + \underbrace{\|\hat{\vec{u}}_k - \Pi_k \vec{u}^{\text{true}}\|}_{\text{regression}}. \quad (17)$$

Reducing k decreases the regression error and increases the truncation error.

The reduction is a net loss Reducing the truncation does reduce the regression error, by 37% (*midacd*), 35% (*full3d*) and 35% (*interface*) — the network does learn a shorter target better, as predicted. But the truncation error grows by factors of 2.9, 2.8 and 17, and the total error approximately *doubles* on all three targets. The *weak* regime degrades likewise. The proposal should be withdrawn.

Why the existing tooling reports the opposite Scored the way `compare_baselines.py` scores, the reduction looks like a clear win: coefficient R^2 rises from 0.9266 to 0.9587 (interface), 0.9660 to 0.9868 (midacd) and 0.9771 to 0.9901 (full13d), and its relative L^2 falls in every case. The reason is that its reference field is $\Pi_k \bar{u}^{\text{true}}$ and not $\bar{u}^{\text{true}}$ — the truth is re-truncated with the model, so the truncation error is invisible by construction. This is exactly the floor-versus-truth distinction of Definition 3, and it is the second place in this note where the distinction changes a conclusion. Any comparison across different k must be scored against the full-resolution field.

On choosing k Since the two terms of (17) move oppositely, the total error is minimised at an interior k . At $k = 64$ the two are comparable on both velocity fields (0.0141 against 0.0175 for midacd; 0.0149 against 0.0119 for full13d), so the deployed truncation sits near the balance point, whereas $k = 24$ is far past it. The predictability limit bounds what is *learnable*, not what is *representable*, and is therefore the wrong criterion for truncation on its own.

7.2 Imposing volume conservation on the interface

The proposal is to impose $\int_D \Delta h \, dx \, dy = 0$ on the predicted interface by projection, on the grounds that the present model satisfies it only approximately and the projection costs nothing.

The constraint is already satisfied With $\bar{w}$ the area weights of Definition 4, the area-weighted mean displacement $\langle \Delta h \rangle_{\bar{w}}$ on the test runs has $\max |\langle \Delta h \rangle| = 6.83 \times 10^{-5}$ m for the *solver's own* fields and 4.37×10^{-5} m for the surrogate: the surrogate violates volume conservation by 0.6 times the solver's own residual, that is, less than the data it was trained on. Imposing the projection makes the residual exactly zero (1.3×10^{-17} m) and changes the relative L^2 error against the full-resolution truth from 0.001550 to 0.001551, a degradation of 0.01%.

Why, and what it confirms $\langle \Delta h \rangle_{\bar{w}}$ is a *linear* functional. Every snapshot nearly annihilates it, hence so does the mean $\bar{g}_h$, hence so does every centred snapshot, hence so does every mode — each being a linear combination of centred snapshots — and hence so does every reconstruction (2), whatever the network predicts. Section 7.2 is therefore a direct confirmation of that argument (Section 8.6 (a) in the exact-mesh case), and the projection should be imposed for the guarantee it provides rather than for any accuracy it buys.

8 Items established for the geometry extension

The natural next step is to let the geometry vary — the anode–cathode distance and the immersion depth — and predict the flow as a function of both the currents and the cell. Everything in this section was established while preparing that step, and is recorded here because each result changed the design of the campaign that would follow.

The setting is parametric model reduction [3], and the central question is how a proper orthogonal decomposition can be defined at all when each geometry is meshed separately: the snapshot vectors then have no common meaning entry by entry. The expected answer was to transfer every run onto one reference mesh. The measured answer is that no transfer is needed, that a resampling grid — the obvious alternative — is worse than it looks for a reason specific to this cell, and that the geometry response is low-dimensional enough for the extension to be affordable. Two constructions proposed in Chapter 2's conclusions are withdrawn as a result, and one correction to the planned campaign design follows from a measurement rather than from an argument.

8.1 Is the transfer construction needed at all?

The reference-mesh construction of Chapter 2’s conclusions — a fixed reference domain, a closed-form map φ , a Piola pull-back and a Helmholtz projection — exists for one reason: the assumption that changing the geometry *regenerates* the mesh, so that snapshots at different (d, s) have different lengths and $\mathbf{G}$ cannot be formed. That assumption is testable for the price of two meshing jobs, and it is worth testing first, because if it is false the entire construction is unnecessary.

The generator does not regenerate; it renumbers The mesh the surrogate lives on is produced by splitting a structured hexahedral mesh into tetrahedra, and the unstructured mesher is invoked only for the external bowls, which are not part of it. Its layer counts are hard-coded constants, independent of the geometry; only the layer thicknesses depend on it. So a change of geometry should not change the mesh combinatorially, and it does not: at two geometries the element count, the node count and the element count of every one of the nine material references are identical.

The *numbering* is not. Dumping the arrays the database actually stores — rather than the merged export, which interleaves them with the bowls — the connectivity disagrees on 29.7% of rows, and the array of material labels differs although its multiset of values does not. A material label is a property of an element, so an identical multiset in a different array can only mean the elements are enumerated in a different order, and a different element order entails a different node numbering. Snapshots from two geometries therefore cannot be stacked as they stand.

A canonical relabelling recovers the correspondence exactly They can be stacked after reordering. The generated mesh is *layered*: it resolves into a fixed number of z -levels, with the same node count on each level at every geometry, the busiest level being the interface plan grid. Ordering each mesh’s nodes by

$$(\text{rank of its } z\text{-level}, y, x)$$

therefore defines the same sequence in both. Tested at the two opposite corners of the intended design — anode–cathode distance 25 against 45 mm, immersion 125 against 175 mm — the induced permutation is a bijection and the connectivity after relabelling is **identical**. Matched nodes are displaced by at most 70 mm vertically, which is exactly the change in bath height, and by a few millimetres laterally, which is the wall cone. The rule uses no geometry at all: no cone law, no layer map, no Jacobian.

Correcting an earlier conclusion in this note An earlier version concluded that a transfer construction was mandatory. That conclusion came from two defective tests and is withdrawn. The first compared the merged export, whose global node list interleaves the cell mesh with the externally meshed bowls, so a change in the bowls shifts every index; the second selected the wrong set of material references, so the node set being compared was itself wrong. With the arrays the database stores, and the ordering above, the correspondence is exact.

Corollary 17 (Relabel, do not transfer). *Two results now combine. Across geometries the meshes are the same mesh differently numbered, and a canonical sort recovers the correspondence exactly (Section 8.1); and transferring to a common physical frame does not improve the decomposition and slightly degrades it (Section 8.5). The consequence for the geometry extension is therefore the opposite of what Chapter 2’s conclusions propose: snapshots at different geometries should be **relabelled and stacked directly**, with no reference mesh, no map φ , no Piola pull-back and no Helmholtz projection. The nodes do move — up to 70 mm vertically*

— so the decomposition remains in a node-index frame in which node k is the same logical node at a different place; but that is precisely the situation the existing campaign already has for interface motion, and Section 8.5 measured that removing it does not help.

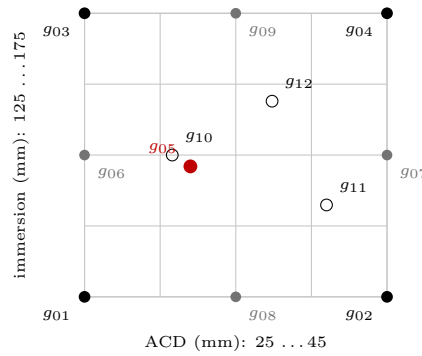
The Piola correction of Section 8.6 remains correct as a statement, and remains necessary for any construction that genuinely interpolates between different discretisations — a mesh-refinement study, or a cell whose plan geometry changes — where a conservative remap [17] rather than nodal interpolation is the right tool. It is not needed for the (d, s) extension.

8.2 Applying the relabelling to solved runs

Corollary 17 is a statement about the mesh *as generated*. The snapshots, however, come out of the stationary module, and two things happen between the two that have to be accounted for before the rule can be used.

8.3 Gate 1: the rank of the geometry direction

Twelve geometries were meshed and solved at uniform current, then relabelled by the rule above and stacked. They are placed in the rectangle $ACD \in [25, 45]$ mm by immersion $\in [125, 175]$ mm: its four corners, the midpoint of each of its four sides, the nominal cell, and three points scattered inside it.



Filled circles are corners, grey are edge midpoints, open are interior fills, and g_{05} in red is the nominal cell — the geometry the existing campaign was run at. Numerically:

	ACD	immersion	
g_{01}	25.0	125.0	corner
g_{02}	45.0	125.0	corner
g_{03}	25.0	175.0	corner
g_{04}	45.0	175.0	corner
g_{05}	32.0	148.0	nominal
g_{06}	25.0	150.0	edge
g_{07}	45.0	150.0	edge
g_{08}	35.0	125.0	edge
g_{09}	35.0	175.0	edge
g_{10}	30.8	150.0	interior
g_{11}	41.0	141.2	interior
g_{12}	37.4	159.5	interior

The corners are deliberate: they are where the geometry change is largest, so a response that is low-rank there is low-rank everywhere between, and a design of interior points alone could not establish that. Three points along each side is the least that can distinguish curvature from a slope.

Where the box comes from It is a choice and not a derivation, and worth stating as such: a band about the nominal cell, $-7/+13$ mm in gap and $-23/+27$ mm in immersion, wide enough that a low-rank verdict means something and narrow enough to stay in a regime the solver is known to handle. Three things bound it, none of which binds at these values.

Resolution. Chapter 1’s mesh study finds that three nodes across the anode–cathode gap “is not a coarse approximation of a smooth field but a failure to represent the layer at all”. The generator scales the layering with the gap rather than holding the layer height fixed, so the gap carries *five* z -levels at every geometry in the box — spacing 5.0 mm at the narrowest corner, 9.0 mm at the widest — and the small end is no less resolved than the nominal cell. That constant layer count is also what makes the relabelling of Section 8.1 possible, so the two facts are the same fact.

Geometry. The anode must remain partly clear of the bath. At 175 mm the box immerses under a third of a real anode (584 mm), so the constraint is not approached.

Operation. A reduction cell is run at a gap of a few centimetres, and the box brackets that range rather than extrapolating beyond it.

What the box is *not* is a validity domain for the surrogate. Nothing here establishes that the response stays low-rank outside it, and the low-rank result of this section is a statement about the interior of this rectangle only. Each solve took between six and eight hours on eight cores, and the nominal geometry g_{05} reproduces the campaign’s own mesh to the byte, which is the check that the generator was driven correctly.

Two questions follow: whether the relabelling holds away from the two corners already tested, and how many modes the geometry direction needs.

The relabelling is exact, at every point of the design box The twelve pilot geometries span the intended design — anode–cathode distance 25 to 45 mm, immersion 125 to 175 mm. Every one produces a mesh with the same node and element counts as the campaign’s own, and twelve different numberings: hashing the connectivity and material arrays of five of them gives five distinct pairs. Section 8.1 tested two corners; the invariance holds over the whole box.

Applying the canonical order to all twelve, on both the velocity mesh and the interface mesh, the connectivity after relabelling is **identical** every time. The interface resolves into a single level, confirming that it is flat before the free-surface loop runs, and its matched nodes are displaced vertically by exactly zero — the datum does not move, because the metal height is held at its nominal value.

The displacements of the velocity mesh decompose as the geometry says they must, which is the strongest evidence that the sort matches the right nodes. Every vertical displacement is the design change itself, and the lateral motion depends on the anode–cathode distance *alone*: identical across the three geometries sharing one value of it, and exactly zero across the three sharing another. Its size is the wall cone applied to the vertical shift, in closed form and agreeing to three figures.

This sharpens an earlier statement. The cone is indeed invariant under both design parameters, but that does not make the *nodes* laterally invariant: a node sits at a fixed layer, the layer’s height moves with the gap, and the invariant cone is then sampled at a different height. Immersion does not enter the layer heights and so moves nodes vertically only.

Each dump was also checked against its own solved mesh, where the two share a numbering and differ only by the free-surface deformation. The discrepancies are tens of millimetres, which is the interface excursion of Section 8.4 and nothing else — metres would have meant the two were not the same node list.

The geometry direction is essentially one-dimensional Proper orthogonal decomposition of the twelve relabelled snapshots, mean removed:

	cumulative energy				modes needed		
	1 mode	2	3	4	99%	99.9%	99.99%
velocity (cuveb)	0.97898	0.99248	0.99880	0.99962	2	4	7
interface h	0.98111	0.99219	0.99939	0.99989	2	3	5

Effective rank 1.04 on both. A single mode carries 98% of the response to a 20 mm change in ACD and a 50 mm change in immersion together.

Gate 1, read honestly The gate was fixed in advance: pass if three modes reach 99.9%, fail if eight or more of the twelve are needed for 99%. Read against the table, the interface passes as written and the velocity misses by six parts in ten thousand. Both clear the failure criterion with two modes against a bar of eight.

The gate should therefore be recorded as passed on its purpose and missed on its letter, and the letter is the part to distrust. A threshold at 99.9% discriminates between two numbers that differ in the third decimal, which twelve snapshots cannot support. What the measurement establishes is what the gate existed to ask: the geometry direction is low-dimensional, and interpolating in it is safe.

Affine in geometry is enough for the velocity and not for the interface Low rank is necessary but not sufficient: Decision 2’s pooled model writes $\mathbf{A}(d, s) = \mathbf{A}_0 + \mathbf{A}_d \delta d + \mathbf{A}_s \delta s$, which needs the modal coefficients to be *affine* in (d, s) , not merely few. With twelve design points a quadratic in (d, s) has six parameters and six residual degrees of freedom, so in-sample R^2 proves little; the column that matters is the leave-one-out predictive Q^2 .

mode	evr	affine		+ quadratic	
		R^2	Q^2	R^2	Q^2
<i>velocity</i>					
1	0.97898	0.9997	0.9995	1.0000	1.0000
2	0.01350	0.9790	0.9491	0.9983	0.9845
3	0.00632	0.0117	−0.9875	0.9934	0.9749
4	0.00081	0.0078	−1.5434	0.9816	0.8483
<i>interface</i>					
1	0.98111	0.9667	0.9368	0.9996	0.9991
2	0.01108	0.0455	−0.9726	0.9625	0.7992
3	0.00720	0.7052	0.2335	0.9789	0.8602
4	0.00050	0.2711	−0.7035	0.8807	0.1627

Q^2 tracks R^2 closely on the quadratic fits of the leading modes, so the second order is real structure and not six parameters chasing twelve points. Weighting each mode by its share of the field variance and counting only out-of-sample skill:

field variance predicted out of sample	affine in (d, s)	quadratic in (d, s)
velocity	0.9913	0.9991
interface	0.9207	0.9953

Geometry is a far easier input than current Set beside the rest of this note, these numbers are remarkable. The response to the 24 currents needs 46–64 modes and a network: a linear map of the currents reaches 0.87 of the field variance and the network 0.98. The response to the two geometry parameters needs *one* mode for 98%, and a six-term polynomial predicts 99.9% of it on a geometry it has never seen.

The asymmetry is not surprising once stated — moving an anode current redistributes the forcing within a fixed domain, while moving the gap or the bath level rescales the domain itself, which is a smoother thing to do — but it has a practical consequence for the campaign. The expensive direction to sample is the current one, not the geometry one. A design that spends its runs on many geometries and few current vectors per geometry is the wrong way round, which is what the foldover of Section 8.7 already implies and what the budget of that section assumes.

The caveat is the same one that runs through this section: every point here is at *uniform* current, so this measures $\vec{g} \mapsto \vec{u}(\vec{g}, \vec{I}_{\text{unif}})$ and not the way the current response itself changes with geometry. The latter is what ϱ of (21) is for, and it is unmeasured.

Corollary 18 (A correction to the campaign design). *The affine expansion in $\vec{g}$ is adequate for the velocity, which it reproduces to 99.1% out of sample, and not for the interface, where it leaves 7.9% of the variance unexplained — an order of magnitude more than the surrogate’s own error. Since the operationally interesting objectives (flatness, local ACD margin, short-circuit risk) are interface objectives, this is exactly where it matters.*

The remedy is cheap. Carrying $\{1, \delta d, \delta s, \delta d^2, \delta s^2, \delta d \delta s\}$ instead of $\{1, \delta d, \delta s\}$ takes the pooled model from $3 \times (24 + 40) = 192$ to $6 \times (24 + 40) = 384$ unknowns per mode, so against the planned ≈ 1800 runs the oversampling falls from $9.4\times$ to $4.7\times$ — thinner, but still comfortable, and the design must simply contain at least six distinct geometries, which the 24-point sweep does many times over. Recording it now avoids discovering it after the campaign.

The mesh itself is a closed-form function of the design The relabelled frame is a node-index frame: the surrogate predicts the field at logical node k , and node k is at a different place in every geometry — up to 70 mm apart (Section 8.3). Placing a prediction in physical space therefore appears to require that geometry’s mesh, which is sixteen minutes of meshing and would defeat a millisecond surrogate.

It does not. Fitting, per node and per component,

$$\mathbf{X}(d, s) = \mathbf{X}_0 + \mathbf{X}_d \delta d + \mathbf{X}_s \delta s$$

over all 401 838 nodes and validating by leave-one-out over the twelve geometries gives a worst-case node-position error of $13.2 \mu\text{m}$ and a median of $11.0 \mu\text{m}$. Elements are tens of millimetres, so this is 5×10^{-4} of one element. Three arrays — a base mesh and two derivatives — reproduce the mesh anywhere in the design box.

Corollary 19 (The map φ , obtained rather than constructed). *Chapter 2’s conclusions call for a map φ from a reference domain to the physical domain, available in closed form, cheap and differentiable, and set out to build one from the geometry. Section 8.3 is that map, and it was not built: it is affine in $\vec{g}$, exact to $13 \mu\text{m}$, trivially differentiable, and it fell out of the relabelling as a by-product. Its structure agrees with the geometry term by term — $\delta z = \delta d + \delta s$, lateral motion $\delta d / \tan \alpha$ depending on ACD alone (Section 8.3) — so it is the analytic map, measured rather than derived, with the layer bookkeeping done by the mesh generator instead of by hand.*

The deliverable of the geometry extension is therefore

$$(d, s, \mathbf{I}) \mapsto (\text{field values at logical nodes, node positions}),$$

both in milliseconds and with no call to the mesher. And the last reason to prefer a fixed resampling grid — that its output is a function on physical space without further work — is removed: so is the relabelled frame’s.

8.4 Incompressibility across geometries

Section 8.6 argued that a transfer between meshes cannot be expected to preserve the discrete divergence, and proposed a Helmholtz–Hodge projection to restore it. With the relabelling there is no transfer, but the objection does not vanish with it: a POD mode is a linear combination of snapshots that each satisfy a *different* discrete divergence operator $\mathbf{B}_g$, because $\mathbf{B}_g$ depends on node positions and those now differ by up to 70 mm. The question is what that costs, and it had never been measured.

The solver’s own velocity is divergence-free to 3.4% For a $\mathbb{P}_1$ field on tetrahedra the gradient is exact and constant per element, so

$$\eta = \frac{\|\nabla \cdot \mathbf{u}\|_{L^2(\Omega)}}{\|\nabla \mathbf{u}\|_{L^2(\Omega)}}$$

is an element sum with no quadrature error of its own. On the fluid submesh (200 925 nodes, 1 080 960 tetrahedra), over eight test runs of the existing campaign, the median is

$$\eta_{\text{solver}} = 3.41 \times 10^{-2}.$$

This is not a defect. The solver assembles SUPG together with a Brezzi–Pitkäranta Laplace term on equal-order $\mathbb{P}_1$ – $\mathbb{P}_1$ [14, 15], so the discrete velocity is never pointwise divergence-free; 3.4% *is* the stabilisation level, and it is the benchmark any statement about incompressibility here must be measured against. Evaluating a run’s values on the reference geometry’s node positions instead of its own changes η by 0.0% (ratio 1.00 on all eight runs): within a geometry, interface motion of a few millimetres contributes nothing.

Stacking twelve geometries costs nothing The same quantity on the geometry pilot, where the nodes move by up to 70 mm. Evaluating g_{01} ’s field on another geometry’s mesh raises η from 3.22×10^{-2} to 3.54 – 3.73×10^{-2} , a 10–16% increase. And for the reconstruction itself — the object that actually matters, each geometry’s field rebuilt from the pooled basis and evaluated on the mesh it claims to live on:

	$k = 1$	$k = 3$	$k = 6$
median η , reconstruction	3.395×10^{-2}	3.417×10^{-2}	3.420×10^{-2}
median η , solver		3.427×10^{-2}	
ratio	0.99	1.00	1.00

A reconstruction built from snapshots on twelve different meshes is as divergence-free as the solver’s own output, on every geometry and at every truncation.

Corollary 20 (The Helmholtz projection is withdrawn). *The projection was proposed as insurance; the insurance is not needed, and taking it out would be a mistake rather than a missed refinement. Imposing $\mathbf{B}_{\text{ref}}\mathbf{u} = 0$ exactly would make the surrogate’s output more divergence-free than the solver it is trained to reproduce — a discrepancy dressed as a guarantee. The mechanism is Section 8.4: the stabilisation residual is an order of magnitude larger than anything the mesh mismatch contributes, so there is no room for the mismatch to matter.*

*This also settles **rem:ml-constraints** on firmer ground than it claimed for itself. The thesis concedes that the linearity argument “degrades with the amplitude of the interface deformation”. It does not degrade, even at 70 mm of node motion, and the reason is that the operator being preserved is itself only satisfied to 3.4%.*

One caveat. These reconstructions use exact modal coefficients, not surrogate predictions; a trained network contributes its own 2–3% field error, which carries its own divergence. That is the same order as the residual and so will not dominate, but it should be measured again on predicted fields once the $(d, s, \mathbf{I})$ surrogate exists.

Why a fixed resampling grid is not the alternative The natural alternative to relabelling is to resample every run onto one grid fixed in physical space. For this cell that fails, and not marginally. The metal–bath interface is not a slightly perturbed plane: over the 11 025 nodes of the interface mesh it spans

$$z \in [0.0803, 0.1666] \text{ m}, \quad \text{span } 86.3 \text{ mm} = 2.7 \times \text{ACD},$$

against a datum `geom_z0` = 0.150 m and a metal pad 150 mm thick. It is a dome: raised some 15 mm in the interior and plunging to 70 mm below the datum at the perimeter, where it meets the sloped walls. The datum is the mean level and not a bound — the area-weighted mean height is 0.15007 m, within 0.07 mm of `geom_z0`, as volume conservation requires — so a grid plane there cuts the interface rather than bounding it, and puts **35.8% of the interface area on the wrong side** (22.61 of 63.21 m²; 4 192 of 11 025 nodes). Entry k of the snapshot vector would then be in liquid metal for one run and in electrolytic bath for another. Since the geometry moves the fluid column top while the datum stays fixed, which entries change material is itself a function of (d, s) : the resampling would inject a geometry-correlated artefact into precisely the matrix it is meant to clean.

The same objection applies at the other free surface. Under an anode the fluid stops at $h + \text{ACD}$, and grid points above it lie inside the anode; that excluded set also moves with (d, s) .

A grid conforming to the interface would answer both — build it in a layer coordinate between the surfaces $z = 0$, $z = h(x, y)$ and the local fluid top, so that the interface is exactly a grid level — but it buys nothing the relabelling does not already give, at the price of a second interpolation error on top of the discretisation error. It is recorded here as the fallback, not the plan.

The resampling machinery works, and is the fallback The fallback was implemented and verified before being set aside, so that it is available if the relabelling should fail on some future mesh family. `post/create_box3d.mac` builds a structured tetrahedral box by `mesh/struct3d.mac` followed by `mesh/hexa.tetra.mac`, the three-dimensional counterpart of `post/create_plane.mac`; and `stat/perform_postpro_ref3d.mac` resamples `fluid_vitesse` and the level set `fluid_phi` onto it by `data/interpolate.mac`, following `post/perform_postpro_midacd.mac`. On an identity test — interpolating a field whose value is the node coordinate itself onto a box strictly inside the source mesh — all 5 551 box points were located and the maximum error was 9.99×10^{-16} , i.e. exact. The construction is sound; it is the *flat* choice of grid, not the interpolation, that Section 8.4 rules out.

Two materials, and what actually matters about them The fluid domain holds two fluids, and it is worth being precise about which of their differences bear on the construction. Their densities differ, $\rho_{\text{Al}} = 2270$ against $\rho_{\text{bath}} = 2130 \text{ kg/m}^3$ (`veloc_rho1`, `veloc_rho2`), and the pressure space is discontinuous across the interface (`ns3d_cont_press` = 0), which is what lets the discretisation carry the jump in ∇p that balances the jump in $\rho \vec{g}$. The viscosity, by contrast, is *not* split by phase: the tensor `fluid_mu_#1..#9` is assembled from `veloc_mud_1,2,3` with no phase index, and the run log confirms a single isotropic value 2×10^{-3} throughout. So the velocity is continuous with continuous gradient across the interface, and the smoothness argument one might expect to make against a non-conforming grid does not apply.

What does apply is Section 8.4, and it is geometric rather than analytic: the objection to a fixed grid is not that the solution is rough at the interface but that the interface is 86 mm from flat, so a fixed grid cannot keep a given entry on one side of it across runs. The relabelling is immune because it never asks where a node is — only which logical node it is — and the material partition travels with the node.

What the invariant counts still buy The negative result is not without consequence for the design. Because the element count, the node count and the material partition are *exactly* preserved, the two meshes are of the same size and resolution, and the difficulty is a relabelling rather than a genuinely different discretisation. A transfer between them is therefore an interpolation between two meshes of equal fineness — the benign case for Section 8.6(b) — and the reference mesh of Phase 1 can be taken to be the mesh of any one geometry rather than a specially refined one. It also means the round-trip test of Phase 1 has a natural null hypothesis: at identical geometry the transfer must be the identity to machine precision.

8.5 Does the transfer reduce the rank? A free pilot says no

Chapter 2’s conclusions expect the reference-domain transfer to *improve* the decomposition, on the grounds that the node-index frame “conflates a change of the field with a change of the mesh”. That expectation is testable without any solver run, because the existing campaign already has a fluid domain whose shape varies: the mesh follows the deformed interface, so node k sits in a slightly different place in every run.

First-order transfer All runs share connectivity and node identity, and the displacement is a few millimetres against a mesh size of tens of millimetres, so the pull-back to the reference positions is

$$\vec{u}_i^{\text{ref}}(\mathbf{X}_{\text{ref}}) = \vec{u}_i(\mathbf{X}_i) + (\nabla \vec{u}_i) \cdot (\mathbf{X}_{\text{ref}} - \mathbf{X}_i) + O(|\delta \mathbf{X}|^2), \quad (18)$$

with the gradient exact per tetrahedron for a $\mathbb{P}_1$ field and averaged to nodes with volume weights. This is not yet the Piola transform: at this stage the map is a small displacement of the *same* mesh, so the Jacobian is the identity to the order retained. The Piola question arises only where the geometry genuinely differs.

How much of a snapshot is mesh motion Over twelve randomly drawn training runs, the correction term of (18) has norm 0.10 to 0.38 of the learned perturbation $\|\vec{u}_i - \vec{u}_{\text{ref}}\|$, median 0.14. The largest values coincide with the largest displacements, up to 65 mm on a strongly deformed run. So the transfer would move about a seventh of a typical snapshot — enough to matter, not enough to dominate.

The transfer does not reduce the rank; it slightly increases it All 1 194 training runs were transferred by (18) and the decomposition recomputed, Δ about the reference run, in both frames:

frame	eff. rank	n_{90}	n_{95}	n_{99}	top 3
n_q = modes needed for a fraction q of the variance					
node-index (as deployed)	13.48	19	24	46	37.6%
reference (transferred)	13.98	19	25	49	36.7%

The node-index row reproduces Chapter 2 exactly (effective rank 13.5, $n_{99} = 46$), which validates the decomposition against the published figures. The transferred row is marginally *worse* on every criterion. The expectation that the rank would fall from 46 to about 30 is not met.

Why, and why it is not a bug The round-trip check confirms the implementation: transferring a run to the reference frame and back reproduces it to a relative L^2 of 3.7×10^{-4} to 1.1×10^{-3} , and *exactly* zero for the reference run itself, where $\delta \mathbf{X} = 0$. The negative result is therefore about the data and not about the code.

The explanation is that the mesh motion is not an independent contamination. The mesh follows the interface, the interface is driven by the anode currents, and the flow is driven by the

same currents — so the apparent “mesh” variation lies in the same low-dimensional response as the field variation it is supposed to be polluting. Removing it does not simplify the solution manifold, and the first-order reconstruction adds a little gradient noise of its own.

What the transfer is for This result is what makes Corollary 17 possible. Had the transfer reduced the rank, it would have been worth its interpolation error even once the relabelling removed the obligation to use it; it does not, so nothing is lost by dispensing with it. The two findings are independent — one says the correspondence can be recovered by sorting, the other that recovering it in a common physical frame buys nothing — and only together do they close the question.

One consequence for the next phase. The mode count for the geometry campaign should be planned at the k the present field already needs, of order 46–64, and not at the ≈ 30 the earlier reasoning suggested: no reduction is coming from the change of frame, because there is no change of frame.

8.6 Corrections to the transfer construction

The transform below is the contravariant Piola map, standard for $H(\text{div})$ -conforming spaces [16]; the point of stating it is that the direction matters and the thesis has it reversed.

Proposition 21 (The Piola pull-back). *Let $\varphi : \hat{D} \rightarrow D$ with $\mathbf{J} = \nabla \varphi$. The contravariant Piola transform pushes a reference field forward by $\vec{u} = \det(\mathbf{J})^{-1} \mathbf{J} \hat{u}$; consequently the pull-back of a physical field to the reference domain is*

$$\hat{u} = \det(\mathbf{J}) \mathbf{J}^{-1} (\vec{u} \circ \varphi), \quad (19)$$

and not $\det(\mathbf{J})^{-1} \mathbf{J} (\vec{u} \circ \varphi)$, which is the push-forward applied in the wrong direction.

The divergence-free guarantee and its three failure modes Discrete incompressibility is a *linear* constraint $\mathbf{B}\vec{u} = \vec{0}$. If a single operator $\mathbf{B}$ is shared by all snapshots, then every mode — being a linear combination of centred snapshots — lies in $\ker \mathbf{B}$, and so does every reconstruction (2). The guarantee fails for three independent reasons. (a) $\mathbf{B}$ depends on node positions, which differ between runs because the mesh follows the interface; the guarantee is exact only for a fixed mesh. (b) Nodal interpolation between non-nested meshes is $O(h^2)$ in L^2 but only $O(h)$ in H^1 ; since div is a first derivative and $\text{div } \vec{u}_h$ is itself a small stabilisation residual, the interpolation error is $O(1)$ *relative to the quantity being preserved*, and the Piola transform, being a change of variables rather than a gradient-controlling projection, does not repair this. (c) With stabilised equal-order elements the solver’s own velocity is not pointwise divergence-free: it satisfies $\int q \text{div } \vec{u}_h = 0$ over the pressure space at best. The target is therefore the solver’s stabilisation level and not zero.

All three are correct as statements about interpolation between different discretisations, which is why they remain on record. None of them bites on the construction actually adopted. Section 8.4 measures that level — 3.4×10^{-2} , an order of magnitude above the initial estimate made here — and finds that a decomposition pooled over twelve differently numbered meshes reaches it exactly. The Helmholtz–Hodge projection proposed as insurance against (a) is therefore withdrawn (Section 8.4); (b) does not arise because nothing is interpolated; and (c), once measured rather than estimated, turns out to dominate the other two by so much that neither can be detected.

8.7 Orthogonal designs for estimating $\mathbf{A}$

The design below is a foldover on the rows of a Hadamard matrix, constructed by the Paley method [11]; foldover and its aliasing properties are standard [10], and the conditioning it buys is what D -optimality maximises [12].

Proposition 22 (Hadamard design). *Since 23 is prime, the Paley construction gives a Hadamard matrix $\mathbf{H}$ of order 24. Let $\mathbf{R} \in \{\pm 1\}^{23 \times 24}$ consist of its rows 2, ..., 24 in normalised form. Then*

1. *each row of $\mathbf{R}$ contains exactly twelve +1 and twelve -1, hence $\mathbf{R}\vec{1} = \vec{0}$, so the constraint $\sum_j p_j = 0$ holds by construction;*
2. *$\mathbf{R}\mathbf{R}^\top = 24\mathbf{I}_{23}$, so the rows are mutually orthogonal and span the constraint hyperplane;*
3. *scaling by $A^* = \min(I_{\text{mean}} - I_{\text{min}}, I_{\text{max}} - I_{\text{mean}}) = 3983$ A is the largest symmetric amplitude that keeps every anode feasible, and it saturates the upper bound exactly.*

All three verified numerically: the Gram matrix has diagonal 24 and off-diagonal exactly 0, and the resulting current vectors satisfy $\sum_j I_j = 490\,000$ A with $\max = 24\,400$ A on the bound and $\min = 16\,434$ A, some 34 A above it. The box is asymmetric about the mean — 4017 A of room below against 3983 A above — so a symmetric design cannot reach both bounds at once, and the binding side is the upper one.

Lemma 23 (Foldover separates odd and even orders exactly). *Let $\vec{c}(\vec{\delta})$ be any map admitting the expansion $\vec{c}(\vec{\delta}) = \vec{c}_0 + \mathbf{A}\vec{\delta} + \mathbf{Q}(\vec{\delta}, \vec{\delta}) + O(\|\vec{\delta}\|^3)$ with $\mathbf{Q}$ bilinear symmetric. Then*

$$\frac{1}{2}(\vec{c}(\vec{\delta}) - \vec{c}(-\vec{\delta})) = \mathbf{A}\vec{\delta} + O(\|\vec{\delta}\|^3), \quad \frac{1}{2}(\vec{c}(\vec{\delta}) + \vec{c}(-\vec{\delta})) - \vec{c}_0 = \mathbf{Q}(\vec{\delta}, \vec{\delta}) + O(\|\vec{\delta}\|^4). \quad (20)$$

Hence running each design row at $+\vec{\delta}$ and $-\vec{\delta}$ makes the estimate of $\mathbf{A}$ unbiased under arbitrary even-order nonlinearity, and yields the quadratic form along that direction at no additional cost. The scalar

$$\varrho = \frac{\|\frac{1}{2}(\vec{c}(\vec{\delta}) + \vec{c}(-\vec{\delta})) - \vec{c}_0\|}{\|\frac{1}{2}(\vec{c}(\vec{\delta}) - \vec{c}(-\vec{\delta}))\|} \quad (21)$$

is therefore an exact measure of the ratio of quadratic to linear response, and is a cleaner nonlinearity probe than a difference of ridge-regularised fits.

Proof. Substitute $-\vec{\delta}$ and use $\mathbf{A}(-\vec{\delta}) = -\mathbf{A}\vec{\delta}$ and $\mathbf{Q}(-\vec{\delta}, -\vec{\delta}) = \mathbf{Q}(\vec{\delta}, \vec{\delta})$; odd terms cancel in the sum and even terms in the difference. $\square$

Sample efficiency, and what it costs Under random Gaussian $\vec{\delta}$ the expected out-of-sample R^2 of $\mathbf{A}$ with $p = 23$ predictors and N runs is $1 - \sigma^2/\text{Var} \cdot (1 + (p+1)/(N-p-2))$, which with the measured $\sigma^2/\text{Var} = 0.185$ is *negative* at $N = 30$ and reaches only ≈ 0.73 at $N = 75$. The 46-run foldover design carries the information of 230–250 random runs. Note that Lemma 23 leaves a cubic bias at full box amplitude; running both ± 4 and ± 2 kA and taking $\mathbf{A}_0 = (4\mathbf{A}_2 - \mathbf{A}_4)/3$ removes it to leading order.

Parameter counting for a pooled model Per modal coefficient, $\mathbf{A}$ has 24 unknowns and a full quadratic form $\mathbf{Q}$ has $\binom{24}{2} + 24 = 300$, so 324 in total. Fitting each geometry independently at fivefold oversampling costs $\approx 1\,620$ runs per geometry; for 24 geometries, $\approx 38\,900$ runs. Positing instead that the coefficients vary affinely in a two-dimensional geometry parameter $\vec{g} = (d, s)$,

$$\vec{c}(\vec{g}, \vec{\delta}) = \mathbf{A}(\vec{g})\vec{\delta} + \mathbf{Q}(\vec{g})(\vec{\delta}, \vec{\delta}) + \dots, \quad \mathbf{A}(\vec{g}) = \mathbf{A}_0 + \mathbf{A}_d \delta d + \mathbf{A}_s \delta s, \quad (22)$$

gives $3 \times 324 = 972$ unknowns per mode for the *entire* campaign. With the screening of Definition 2 retaining 40 products this is $3 \times (24 + 40) = 192$ per mode, i.e. about ninefold oversampling against a budget of $\approx 1\,800$ runs.

Two corrections to that count, both from measurements made after it was written. The affine dependence on $\vec{g}$ assumed in (22) is *not* adequate for the interface (Corollary 18): carrying the three second-order geometry terms as well doubles the count to 384 per mode and reduces the oversampling to $4.7\times$, which is the figure to budget against. And the counts 24 and 300 are the sizes of the design blocks, not of the identifiable parameter sets: by Proposition 15 only 23 directions are ever exercised, so the true counts are 23 and 276. Using the larger numbers makes the oversampling estimate conservative, which is the safe direction for a budget.

What remains untested is the premise itself — that $\mathbf{A}$ and $\mathbf{Q}$ vary smoothly in $\vec{g}$ at all. Section 8.3 measures the geometry response at uniform current and finds it low-dimensional, but that is the response and not the *map*; establishing the latter needs current perturbations at several geometries, and (21) is the instrument.

8.8 What the campaign costs, and what fits

The design of Section 8.7 is affordable only against a stated budget, and the numbers below are measured rather than estimated: the scheduler prices a production-sized job — eight tasks, eight hours, which is what the twelve pilot solves actually took — at **CHF** 0.35, and the twelve consumed 57.4 core-hours each.

A sweep geometry is 46 foldover runs, one base run at uniform current and seven interior points, so 54; an anchor geometry runs the foldover at two amplitudes to cancel the cubic bias of Lemma 23, so 93. Adding the ϱ gate and one held-out geometry:

configuration	runs	CHF	against the allocation
24 sweep, 2 anchors, ϱ on a cross of five	1 837	643	−177
24 sweep, 2 anchors, ϱ on three points	1 752	613	−147
20 sweep, 2 anchors, ϱ on a cross	1 621	567	−101
16 sweep, 2 anchors, ϱ on a cross	1 405	492	−25
16 sweep, 2 anchors, ϱ on three points	1 320	462	+4
16 sweep, 1 anchor, ϱ on three points	1 227	429	+37
the ϱ gate alone, on a cross of five	235	82	+384
the ϱ gate alone, on three points	150	53	+414

The allocation is a cap of CHF 1 000 on the account, of which CHF 534 is already consumed, leaving **CHF** 466 at the time of writing. The campaign as designed does not fit, and the two configurations that do fit spend the entire remainder, leaving nothing for a re-run, a failed job, or any other work on the account for the rest of the period.

What shrinking the sweep costs Not identifiability — the pooled design remains full rank down to 16 sweep geometries — but conditioning. Building the design matrix for the quadratic geometry model of Corollary 18 and taking its singular values:

sweep geometries	oversampling	condition number
24	$3.9\times$	438
20	$3.3\times$	639
16	$2.8\times$	2 703
14	$2.5\times$	— rank deficient

The degradation is not gradual. Going from 24 to 20 costs a factor 1.5 in conditioning; from 20 to 16 costs a factor 4.2; and at 14, or at 16 with the interior points cut from seven to five, the design stops being identifiable at all rather than becoming gradually worse. The cliff sits immediately below where the budget forces the design.

Recommendation Run the ϱ gate first and decide afterwards. It costs CHF 82, under a fifth of what remains, and it answers the question on which the other CHF 535 depends — whether **A** varies smoothly enough in $\vec{g}$ for the pooled model to be legitimate at all. It also measures the one quantity the budget silently assumes and nothing has checked: the convergence rate at the corners of the design box under a full-amplitude current perturbation. Every pilot run converged, but every pilot run was at uniform current; if corner acceptance is 80% rather than the $\approx 100\%$ the nominal-geometry campaign enjoyed, every figure in the table above is optimistic by a quarter.

If the gate passes, the honest options are to raise the cap or to accept 16 sweep geometries at a conditioning of 2 700 and no contingency. Of the two, raising the cap is the better engineering: CHF 643 against a cap of CHF 1 000 already half consumed is not an unreasonable request, whereas the alternative spends everything on a design sitting one step above rank deficiency.

Storage Not a constraint. The campaign master and per-run exports run to about 1.1 GB per run, so 1 800 runs is some 2 TB on top of the existing 1.9 TB. The filesystem has over a petabyte free and no project quota is enforced.

9 Summary of what is established and what is not

Four statements carry the rest, and they are easier to hold together than the list below.

First, the network is needed for prediction — of the velocity and the interface, not universally. It beats a linear map in every sampled regime and by the widest margin when the cell is least ordinary, which is when a model is most wanted. But on the metal current density a *linear* field model scores 1.000 and beats it, because there the linear residual is two parts in a thousand rather than one part in eight. What decides the matter is the residual, not the model class.

Second, the quantities an engineer reads should be computed from a predicted field and never regressed on the currents directly. Direct regression is worse than predicting the mean on every velocity functional. This too is governed by the residual, and it is subject to the same bound as everything else: it holds inside the sampled regimes and fails outside them.

Third, the network should *not* be the thing an optimiser sees. With the affine field model every objective considered here is strictly convex with a unique optimum reached from every start; the irreproducibility measured on the network is its own artefact. The flexibility that makes it the better predictor is what makes it the worse objective, because where the data does not constrain the map the network is free, and that freedom appears downstream as structure that is not in the physics.

Fourth, the campaign’s coverage of rare operating conditions is load-bearing and its regime mix is not an imbalance to be corrected. Removing a condition from training breaks every model class equally.

Established, on existing data, with no new solver runs.

- Functionals of a *network*-predicted field beat direct regression on every L^1/L^2 target (0.97–0.99 against 0.80–0.94), and rescue the cases where direct regression collapses to $R^2 < 0$; a *linear*-predicted field does not (Table 1). The implementation agrees exactly with the solver on every L^∞ functional (Section 2.4).
- The residual of the best predictor is regression error, not basis error: the truncation floor is ≈ 1.000 on every L^1/L^2 target (Table 2).
- The metal current density has effective rank 5.9 and is affine in the currents to $R^2 = 0.9982$; the metal-pad-instability proxy built from it is recovered at $R^2 = 1.000$ by a *linear*

field model and at -0.107 by direct linear regression, so its nonlinearity is a property of the observable and not of the cell (Proposition 9).

- Across 30 functionals spanning five planes, three fields and three continuity classes, predictor (iv) is best on 28; and the behaviour is predicted by the functional’s class rather than measured case by case — affine observables gain nothing from a field model (Proposition 7, confirmed to 0.009), absolute observables are recovered by a linear one, quadratic observables require an accurate one (Corollary 8).
- Direct regression of an observable is not merely worse than evaluating it on a predicted field — on a cell with an anode being replaced it is *negative* where the field route returns 0.78, while in ordinary operation the two are within three percent of each other. The advantage is concentrated in the conditions a model is actually consulted for (Section 2.5).
- The network is better than a linear map in every regime the campaign samples, by factors of two to fourteen, and its margin is largest where the cell is least ordinary — 2.4% against 11.8% during an anode replacement. Remove a regime from training and no predictor of any class beats the mean on it, so the result bounds what a campaign must *cover* rather than which model to use (Section 3).
- $n \approx 400$ –600 runs suffice for the linear operator at a fixed geometry; the screened quadratic map has a stability threshold near $n = 600$ rather than a smooth curve (Measurements 6, 6).
- rank $\mathbf{A} = 23$ exactly, its spectrum is computable from a 64×24 matrix (Lemma 14), and its conditioning predicts the reproducibility of the inverse design problem measured independently by multi-start optimisation. That irreproducibility belongs to the surrogate and not to the cell: with the affine field model every objective used here is strictly convex and has a unique optimum, reached from every start (Proposition 16). The corollary for inverse design is to optimise the convex model inside a trust region and let the network place the region (Section 5).

Refuted, having been measured.

- That a *linear* field model suffices for functionals: column (iii) of Table 1 scores 0.03–0.73 and loses to direct quadratic regression almost everywhere.
- That retraining at the predictability truncation $k \approx 24/24/7$ improves the surrogate: the total error against the full-resolution field approximately doubles on all three targets, because the truncation error grows faster than the regression error falls (Section 7.1). The opposite conclusion follows from the existing tooling only because it re-truncates the reference field with the model (Section 7.1).
- That the predicted interface violates volume conservation appreciably: it violates it by 0.6 times the solver’s own residual, and imposing the projection changes the error by +0.01% (Section 7.2).
- That varying the anode–cathode distance within the constant-ACD regime reintroduces the interface–gap–current feedback: that regime preserves the gap pointwise at *every* design ACD, so the feedback belongs to the fixed-anode regime alone.
- That immersion depth is uniquely problematic for a layer map: the wall cone $\mathbf{geom_xb}(z) = W_x + z/\tan \alpha_x$ is invariant under both geometry parameters, the ACD dependence cancelling identically.

Also refuted, by Tier C. That the horizontal current in the metal is evidence for a nonlinear surrogate (Section 2.7). And a latent defect found in passing: `normalize: false` in the deployed configurations is calibrated to velocity-scale coefficients and fails silently on a field with different units (Section 2.7).

Established for the geometry extension.

- The mesh generator does *not* preserve node numbering under a change of (d, s) , but it does preserve everything else: over the whole design box the twelve pilot geometries give the same 401 838 nodes and 2 109 072 elements with twelve different numberings (Measurements 8.1, 8.2). Ordering by (level rank, y, x) recovers the correspondence exactly, so snapshots at different geometries are stacked by **relabelling**, with no reference mesh, no map φ , no Piola pull-back and no interpolation (Corollary 17).
- The relabelling must be computed from the mesh as generated, not from the solved mesh, whose level structure the free-surface loop destroys; the dump that makes this possible costs about ten minutes per geometry and requires no run to be repeated (Section 8.2).
- Carried out at all twelve pilot geometries, on both the velocity mesh and the interface mesh, the relabelling gives **identical connectivity** every time, and the matched-node displacements are the design changes in closed form: δz is $\delta d + \delta s$ exactly, and the lateral motion is the cone law $\delta d / \tan \alpha$ to three figures, depending on ACD alone (Section 8.3).
- **The geometry direction is essentially one-dimensional:** one mode carries 97.9% of the velocity response and 98.1% of the interface response over the whole design box, effective rank 1.04, two modes reach 99.2% (Section 8.3). Gate 1 passes on its purpose; the velocity misses its written threshold in the third decimal place, which is not a distinction twelve snapshots can support (Section 8.3).
- But *affine* in (d, s) is enough only for the velocity. Measured by leave-one-out, the affine expansion predicts 99.1% of the velocity variance out of sample and only 92.1% of the interface variance; adding the three second-order geometry terms gives 99.9% and 99.5% (Section 8.3). Decision 2's pooled $\mathbf{A}(d, s)$ therefore needs a quadratic in $\vec{g}$, at $4.7\times$ rather than $9.4\times$ oversampling (Corollary 18).
- Incompressibility survives the stacking untouched. The solver's own velocity is divergence-free only to $\eta = 3.41 \times 10^{-2}$ — SUPG plus a Brezzi–Pitkäranta term on equal-order $\mathbb{P}_1\text{--}\mathbb{P}_1$ — and a POD reconstruction pooled over twelve different meshes reaches the same 3.42×10^{-2} at every truncation (Measurements 8.4, 8.4). The Helmholtz projection proposed as insurance is therefore **withdrawn**: it would make the surrogate more divergence-free than the solver it reproduces (Corollary 20).
- The mesh itself is predictable: $\mathbf{X}(d, s)$ is affine to $13\ \mu\text{m}$ worst case by leave-one-out over all 401 838 nodes, so the surrogate returns field values *and* node positions in milliseconds with no call to the mesher — and this is the closed-form φ Chapter 2 set out to construct (Section 8.3, Corollary 19).
- The alternative — one grid fixed in physical space — is ruled out for this cell, and geometrically rather than analytically: the interface spans 86 mm, 2.7 times the ACD, so a plane at the datum would put 35.8% of the interface area on the wrong side of it, and which entries change material is itself a function of (d, s) (Section 8.4, Section 8.4). The resampling machinery was nevertheless implemented and verified to machine precision, and is kept as the fallback (Section 8.4).

Established by composing the two surrogates.

- Substituting the network’s flow for the solver’s changes the published alumina homogeneity metric by 0.25%, against a velocity error of 0.59%: the transport attenuates surrogate error rather than amplifying it (Section 10).
- Across operating conditions the same metric moves by -14% to $+7\%$ and the alumina field by up to 9.7% — ten to forty times what predicting the flow costs, so the effect is comfortably resolved. It is not monotone in the size of the current disturbance, and the only condition that lowers the minimum concentration is an anode being replaced.
- A synthetic perturbation of the flow, used before the real residual was available, over-estimated the sensitivity by a factor of thirty. Proxy perturbations are not substitutes for measured residuals.

Not yet established. The functionals built from `fields_full/{forces,induction}`, which Chapter 2 measures to have coefficients of variation of 0.30% and 0.22% and which are therefore nearly invariant across the campaign. And everything that requires currents and geometry to vary *together*: the twelve pilot runs are all at uniform current, so they measure the geometry response alone. Whether **A** itself varies smoothly across the design box — the nonlinearity ratio ρ of the plan’s step 1.2 — needs the current perturbations at several geometries and is the next measurement, not this one.

10 The question this opens

Everything above treats the surrogate as the end of the pipeline. It is not. A velocity field is an intermediate, and Section 2 is really the first case of a more general question: what happens when a predicted field is handed to something downstream. There the something was a functional. In practice it is another solver.

The chain, and who has surrogated which link Alumina is injected by a number of feeders, transported by the flow, dissolved, and consumed at the anodes; local deficits cause anode effects and local surpluses cause sludge, so its homogeneity is an operational target. The chain is

$$\text{anode currents} \longrightarrow \text{MHD flow} \longrightarrow \text{dissolved alumina}.$$

Patouillet et al. [18] surrogate the second link with the same solver used here: a graph neural network maps seven feeder mass-flow fractions, under a conservation constraint, to the dissolved-alumina field on the mid-ACD plane, trained on some fifteen percent of 1 443 admissible combinations and predicting in about a second. This note surrogates the first link. The two studies hold still precisely what the other moves — their currents are assumed uniform across the anodes, and the feeders here are fixed — so between them the chain is covered exactly once, with no overlap and no composition.

The question *Given a chain of physical models, where should the surrogate boundary be drawn, and can the answer be predicted rather than discovered?* Three configurations are available and they are not obviously ranked:

1. **end to end**: learn (currents, feeders) $\mapsto$ alumina as one map, so the flow never appears;
2. **composed**: predict the flow, then run the alumina solver on the predicted flow — the field-then-operator route, with the operator a solver rather than a functional;
3. **hybrid**: surrogate only the expensive link and keep the cheap one exact.

Why the answer is not obvious here Because the two links have opposite structure. Alumina transport is *linear in its source*: the concentration is an advection–diffusion response to injection, so superposition holds in the feeder rates up to dissolution kinetics and saturation. But it is *nonlinear in the velocity*, which enters through the advective term $\vec{u} \cdot \nabla c$. So the link that looks benign from the feeder side is the one that amplifies an error in the flow, and a composed pipeline inherits a nonlinearity that neither half exhibits on its own.

The instrument already exists Proposition 6 and the rule of Section 2.3 were derived for a quadratic functional, but nothing in them is specific to functionals: the error a downstream operator inherits is governed by ϱ , the field model’s residual, and by that operator’s sensitivity to it. Generalising from functional to solver is the natural next step, and the classification of Corollary 8 — affine observables gain nothing, absolute observables need any field, quadratic observables need an accurate one — is the special case that has already been verified, on thirty of them.

What it would take Little that does not exist. The coupling is wired: the alumina module reads the stationary solution directly, on the same mesh as the existing campaign, so the 1 594 runs already in hand drive it with no new magnetohydrodynamic simulation. The composed pipeline is then a surrogate feeding a solver that runs at about real time. The end-to-end control requires the same runs and a second training. What is genuinely new is only the comparison, and the attempt to predict its outcome in advance from ϱ rather than to report it afterwards.

A first measurement

The composed route was built and run, so the question above is no longer entirely open. The alumina module reads its velocity from the stationary database; a gated override in `alumin/load_data.mac` lets a surrogate field be substituted in its place, with everything else held identical. Two experiments follow.

What composing costs For one held-out run, the alumina model was run twice: once on the solver’s own flow, once on the network’s prediction for the same currents.

	solver’s flow	surrogate’s flow	change
mean dissolved alumina (wt%)	2.73811	2.73357	−0.17%
σ , the homogeneity metric	0.40790	0.40687	− 0.25%

A velocity error of 0.59% produces an error of 0.25% in the published metric: the transport *attenuates* the surrogate’s error rather than amplifying it, by a factor of about 0.4. For comparison, the deep-learning surrogate of [18] reports a validation error of 6.25% on the same quantity, so composing is far from the limiting term.

This overturns a proxy experiment done first, in which the flow was perturbed by a smooth random field of controlled amplitude and the metric appeared to be amplified some fourteen-fold. That perturbation behaved like an added large-scale circulation, which changes transport globally; a real surrogate residual is structured, concentrated where the network is weakest, and largely averages out over a feeding cycle. The lesson is worth carrying: a synthetic perturbation is not a substitute for the actual residual, and here it was wrong by a factor of thirty.

What the composition buys Running the pipeline across five operating conditions, from uniform current to a disconnected anode:

condition	current spread	σ (wt%)	min (wt%)	alumina field vs uniform
uniform	0%	0.3798	1.982	—
gaussian	10.7%	0.4069	2.038	3.35%
single	21.3%	0.3252	2.121	7.14%
weak	70.9%	0.3951	1.953	7.57%
dead	108.3%	0.3563	2.144	9.70%

The alumina distribution moves by up to 9.7% as the current distribution changes, and the homogeneity metric by -14% to $+7\%$ — ten to forty times the 0.25% that predicting the flow costs. The effect is comfortably resolved by the surrogate, which is the condition for the composition to be worth building.

Two features of the table are worth more than the headline. The response is *not* monotone in the size of the disturbance: **single**, at a fifth of the spread of **dead**, moves σ further and in the opposite direction. It is the pattern of the current and not its magnitude that decides where the alumina goes, which is exactly why the current distribution has to be an input rather than a correction factor. And **weak** is the only condition that lowers the *minimum* concentration below the uniform case. Local deficits are what precipitate anode effects, so the condition that thins the floor is a cell around an anode replacement.

What this does not show That the *optimal feeding* changes. Establishing that requires running an optimisation over the feeder settings at each current distribution and comparing the optima. What is established is the precondition: the quantity being optimised moves with the current distribution by far more than the surrogate’s own error, so the question is answerable and worth asking. Section 11 reports the attempt, which found the parameterisation already present in the solver and then failed on an instrument fault rather than on the physics.

And one engineering consequence An optimal feeding strategy computed at uniform current is optimal for one flow. The current distribution is not constant — it drifts with anode age, and it is most disturbed exactly when an anode is replaced, which is also when the alumina field is most at risk. Only a composed model can ask how the optimal feeding moves when the cell changes, and Section 3 has already measured that an anode replacement is where a surrogate’s margin over a linear model is widest.

11 The feeding question, pursued

Section 10 closed by saying that showing the *optimal* feeding moves would need the feeder parameterisation of [18]. It does not: that parameterisation is in the solver this note already uses. The attempt was made, and it failed, but not on the physics. It failed because the alumina objective does not converge on the mesh the campaign’s velocity fields live on. That is the result of this section, and it bounds what the composed pipeline can be asked until it is repaired.

Notation, against the two sources The two works this section builds on measure homogeneity differently, and the difference is not cosmetic. [19] minimises

$$J = \frac{1}{\Delta T |\Omega_{\text{el}}|} \int_{T-\Delta T}^T \int_{\Omega_{\text{el}}} (c(x, t) - \bar{c}(t))^2 d\Omega dt,$$

the spatial variance of the dissolved alumina concentration c over the bath volume Ω_{el} , averaged over one feeding period ΔT . [18] report instead σ , the standard deviation of the period-averaged concentration *on the mid-ACD plane*, in weight percent. These are not the same quantity on the

same domain and $\sigma \neq \sqrt{J}$: the first is a volume average of an instantaneous variance, the second a planar standard deviation of a time average. The five-condition sweep of Section 10 reports σ and is comparable with [18], whose uniform-feeding reference is 0.55 wt%; the convergence measurements below report J and are comparable with [19]. Both are stated in the units of their source and neither is converted into the other.

The design variables also differ, and the symbols are theirs. [18] optimise $f_{Qm,i}$, the fraction of the nominal mass flow rate assigned to feeder i , which at a fixed dose is an injection frequency. [19] optimises the load weights w_i at a fixed period, under $\sum_i w_i = M$ with M the mass consumed per period; the solver normalises the same weights to $\sum_i w_i = 1$ and carries the mass rate separately, so the constraint is written that way here.

11.1 The parameterisation was already present

Nothing had to be obtained from [18]: their design variable is a case parameter of the solver used here. The `alumin` module exposes, per feeder, a position, a weight, a first injection time and an injection period. The CNG2 case drives all of them from one variable per feeder, `f_Qm`, taking values in $\{0, \frac{1}{2}, 1, 2\}$ and setting the period to $84\text{s}/\text{f_Qm}$ at a fixed dose; it is therefore an injection-frequency multiplier. Requiring the total feed rate to match consumption fixes $\sum \text{f_Qm} = 7$ over the seven feeders, which admits 1443 configurations — the same number [18] report for their admissible set.

The thesis that produced this module [19] settles three further points and saves the corresponding experiments. Its objective J is what the solver already exports. Its design variable is the load weight w_i at a fixed period rather than the frequency $f_{Qm,i}$, and in w the objective is strictly convex, with Newton converging from any start. And two knobs are dead: the feeding times move the objective by under five percent, and the optimal period is always the shortest technically admissible one.

11.2 What the objective costs to measure

Cyclic rotations of the feeder firing order are exact time shifts of one another at the periodic state, so the spread of the objective across them is numerical error and nothing else. Over seven rotations on the 261 402-node mesh the peak-to-peak spread is 0.04% at sixty feeding cycles and 0.49% at twenty-five; [19] measures 0.5–1.8% for the same test. One evaluation costs 1.48 core-hours, so a design of a few hundred points is affordable at the scale of a few francs. The instrument is quiet enough and cheap enough; the difficulty is elsewhere.

11.3 Two results withdrawn

An enumeration of 211 mass-balanced frequency configurations on two current distributions, 436 runs, completed without a single job failure and produced a clear answer: the optimum moves, worth 70.8%, with a rank correlation of 0.20 between the two landscapes. The answer is void. Only 43 and 53 of the 211 configurations had converged; the median drift of the objective over the final two periods was 2.4–2.7% against a floor of 0.4–0.7%, and the ranking was sorting configurations by how slowly they settled.

Three configurations from that campaign, rerun to 15 122s, then showed one schedule 22.9% better than current practice under uniform current and 61.3% worse under an anode replacement, at drifts of -0.51% and $+0.57\%$. That too is withdrawn, for a reason worth stating as a measurement in its own right: *in this solver a small drift does not mean a converged quantity*. Four configurations relaxed from a converged state each passed through a turning point where the drift fell to 0.05%, indistinguishable from a limit, and then moved by 19, 30 and 38 percent; two crossed from better than nominal to worse during the relaxation. A

convergence test read at one instant cannot separate a turning point from a limit, and the test used here was exactly that.

11.4 The mesh

The same nominal configuration, with a velocity and a geometry taken from the same stationary solve, was run for 30 240 s on each of the two meshes in use. On the 261 402-node mesh the objective reaches its periodic value by about 6.5 ks and then does not move: 0.095773 at every window from 6 552 to 29 064 s, a total movement of +0.03% between five and thirty kiloseconds. On the 401 838-node mesh the same case rises by 28% between 3.5 and 15 ks and is still rising at twenty, non-monotonically. The case files are identical between the two; only the mesh and the stationary database differ, and substituting one run’s velocity onto another run’s deformed interface was separately excluded as a cause by running a fully self-consistent case, which behaves like the rest of the fine-mesh family.

Every quantity measured on the finer mesh is therefore unsafe, and the two sections above are withdrawn on that ground as much as on the convergence test.

The fine mesh was under-stabilised, and the missing term is isotropic rather than streamline. The convection–diffusion equation is solved with SUPG, which stabilises along streamlines only; the case carries a streamline coefficient of unity and *no* isotropic term at all. Streamline stabilisation does not control crosswind oscillation, and on fine cells at high cell Péclet number that is what fails. Running the nominal case to 15 ks on the fine mesh while varying the two coefficients separates them cleanly:

streamline	isotropic	J at 5 ks	J at 15 ks	change
1.00	0.00 (shipped)	0.1890	—	diverging
2.00	0.00	0.2008	0.2581	+28.5%
4.00	0.00	0.1994	0.2505	+25.6%
1.00	0.05	0.1336	0.1343	+0.5%
1.00	0.20	0.0763	0.0686	−10.1%

Quadrupling the streamline coefficient changes nothing: the objective still climbs by a quarter over the same window. Adding five percent of isotropic stabilisation makes it flat. Twenty percent over-damps it, driving J below the coarse-mesh value and still falling. The diagnosis is therefore confirmed, and the coarse mesh converged with the shipped settings because its larger cells sit at a lower Péclet number, where streamline stabilisation alone suffices.

Confirming the mechanism does not make the number trustworthy. The converged value depends on the isotropic coefficient — 0.1343 at five percent against 0.0686 at twenty, a factor of two from a numerical parameter, with the coarse mesh at 0.0958 between them. Absolute J on this mesh is therefore not a quantity to report. What matters for the question of Section 10 is weaker: whether the *ordering* of feeder configurations is stable even when the level is not. Four weight configurations run at two isotropic coefficients give the same ordering, with a Spearman correlation of 1.000 and a level shift of −9.7%. That is robustness to the numerical parameter, and it is not the same as convergence: both arms are read at the same time, and could be wrong together.

They are. Judged by movement between 5 and 15 ks rather than by instantaneous drift, only the nominal configuration is settled (+0.5%); the perturbed ones move by 20, 51 and −11 percent while their drift sits between 0.02 and 0.13%. The isotropic term converges the case the mesh was tuned on and not the cases the study needs.

11.5 The coarse mesh is the usable instrument

The same three perturbed configurations on the coarse mesh converge completely, with the stabilisation left as shipped:

configuration	J at 5 ks	J at 30 ks	change	drift	mean c (wt%)
nominal	0.095744	0.095773	0.0%	0.00%	2.7601
5	0.090161	0.090175	0.0%	0.00%	2.7568
17	0.087873	0.087912	0.0%	0.00%	2.7554
31	0.094740	0.094817	0.1%	0.00%	2.7518

Every configuration is flat to a tenth of a percent over twenty-five kiloseconds, where the same configurations on the fine mesh moved by twenty to fifty-two. The mean dissolved concentration spans 0.3% across the four against 7% on the fine mesh, so the confound noted in Section 11.4 — a configuration scoring well by dissolving less — is also largely a fine-mesh artefact.

The two meshes rank the configurations in opposite orders. On the coarse mesh configuration 17 is the best, at 8.21% below nominal; on the fine mesh it is the worst, at 126% above. The agreement between the two fine-mesh stabilisation settings was therefore two wrong answers agreeing, exactly as the caveat above allowed.

Two things make the coarse-mesh numbers worth believing where the fine-mesh ones were not. They are converged on a test that excludes a turning point, and they are the right size: 1.00, 5.85 and 8.21 percent against nominal, inside the 1.1 to 7.4 percent that [19] reports for the same design variable. Against the 0.04% rotation floor of Section 11.2 these are twenty-five to two hundred times the noise. The instrument works; it is the other mesh that does not.

11.6 What this blocks

The surrogate predicts velocity on the 401 838-node mesh, because that is the mesh the campaign was run on. The alumina module converges on the 261 402-node mesh. Until the stabilisation question is answered the composed pipeline has no mesh on which both halves are sound, and the measurement proposed in Section 10 — how much field accuracy a downstream solver needs — cannot be made. Repairing the fine mesh was the cheapest of the three available routes, and it converges the nominal case but not the others. The remaining routes both end at the coarse mesh, which does converge: interpolate the campaign’s velocity fields onto it, or run a stationary campaign there directly. The first is the cheaper and is what the composed measurement now waits on.

It is worth separating what this does and does not cost. The physics question of Section 10 is untouched: the precondition measured there, that the quantity being optimised moves with the current distribution by far more than the surrogate’s own error, was measured on the mid-ACD plane and does not depend on the long-time behaviour of the bath field. What is blocked is the step beyond it, from a quantity that moves to an optimum that moves.

A How every quantity in this note is measured

A number is only as good as the population, split and weighting behind it, so those are fixed once here and not restated. Departures from this protocol are flagged where they occur.

Run populations. Four counts appear, and they are different:

1 600	rows in <code>manifest.csv</code>	the campaign as designed
1 594	groups in <code>master_ml.h5</code>	the campaign as accepted
1 525	rows in <code>scalars.csv</code>	the accepted runs excluding <code>dead</code>
163	the test split	what every held-out score is computed on

Six designed runs are absent from the master — `stat_0060` and `stat_0723` (`gaussian`), `stat_1479`, `stat_1483`, `stat_1487` and `stat_1525` (`weak`) — so the `weak` regime has 76 runs and not 80,

which matters wherever `weak` carries an argument. The 69 runs in the master but not in `scalars.csv` are *exactly* the 69 dead-anode runs, which is what “bulk” means throughout. The regime census of the master is `gaussian` 1198, `gradient` 100, `cluster` 80, `weak` 76, `single` 70, `dead` 69, `uniform` 1.

The split. One split, stratified by regime with seed 42, giving 1 194/237/163; it is identical for all three targets, so a run is in the test set for every one of them. The basis $\mathbf{V}_k$, the mean $\bar{g}_h$ and the target statistics are fitted on the training rows alone, because `apply_pod` is called after the split with `fit_idx = train_idx`. The Δ -learning reference is `stat_0001`, the single `uniform` run, held fixed rather than re-detected.

Quadrature. Φ uses mass-matrix weights $\vec{w}$ normalised to $\sum_i w_i = 1$: nodal volumes from the tetrahedra for `full3d`, nodal areas from the triangles for `interface` and `midacd`. Element volumes on this mesh vary by orders of magnitude between the thin ACD gap and the metal pad, so a node average is not a volume average and the distinction is not cosmetic — it is the 1.293 factor of Section 2.4.

The weights are computed once, from the reference run’s mesh, and reused for every run. They are therefore a fixed quadrature rather than the exact volume integral on each run’s own deformed mesh. This is deliberate: it keeps Φ one fixed quadratic form, so that a difference between two predictors is a difference in the field and not in the rule applied to it. The cost is bounded by the mesh motion, a few millimetres against elements of tens of millimetres.

What R^2 means here, and it is not one thing. Unless a table says otherwise, every R^2 is **held out on the 163-run test split**. But two conventions are in use, because scalars and fields are not scored the same way, and quoting one against the other would be misleading:

on a scalar functional equation (10), over the 163 test runs with $\bar{\Phi}$ the test-set mean. This is the convention of Tables 1, 3 and 4, and a negative value means worse than predicting the mean — which several predictors are.

on a field or a coefficient vector a single pooled ratio over *every entry* of the matrix, $1 - \|\mathbf{Y} - \hat{\mathbf{Y}}\|_F^2 / \|\mathbf{Y} - \mathbb{E}_{\text{rows}} \mathbf{Y}\|_F^2$, centred column by column. This is the convention behind the field-level figures quoted for the surrogates (0.977, 0.815, and so on).

The two are not comparable: the first asks how well one number is predicted, the second how much of a 602 775-dimensional variance is captured. A field score of 0.88 and a functional score of 0.49 can describe the same model, and in Section 2 they do.

The exceptions to the held-out rule, all labelled at the point of use: Section 2.2 and Section 2.4 are over all runs rather than the test split, since neither involves a fitted model; and the geometry results of Section 8 are leave-one-out over 12 points, there being no split to hold out.

The five diagnostic planes. Positions follow the solver’s source and the CNG2 geometry (`metal_height` = 0.15 m, `ACD` = 0.032 m): the horizontal plane at $z = 0.150$ m, and the four cross-sections at $x = \pm 5.6056$ m and $y = \pm 1.14333$ m.

Truncation of the sensitivity operator. The least-squares solve uses `rcond` = 10^{-4} . The window is bounded below by $\sigma_{24}/\sigma_1 = 3.3 \times 10^{-6}$, the null direction created by the fixed total current, and above by $\sigma_{23}/\sigma_1 = 0.53$, the smallest genuine direction; 10^{-6} , which reads as a safely small threshold, falls below the null value and truncates nothing. With 10^{-4} in place $\sigma_{24}/\sigma_1 \leq 4.4 \times 10^{-17}$ on all three targets.

Which field. Every functional is applied to one of three fields, and which one is always stated: the solver’s own at full resolution (the truth), the same field through the POD basis (the floor), or a predicted field. Where an error is reported against a field rather than a functional it is the relative L^2 norm against the *full-resolution* field, never against a re-truncated one — an error that Section 7.1 shows is easy to make and reverses a conclusion.

B Reproduction

result	command
Tables 1, 2	<code>field_functionals.py --target {midacd,full3d,interface}</code>
Section 2.4	<code>join data/functionals_T2_*.npz to scalars_cluster.csv</code>
Table 5	<code>prepare_alucell.py --train-modes gaussian</code> then <code>train.py</code> and <code>compare_baselines.py</code>
Table 8	<code>SVD of lstsq(train/P, train/U, rcond=1e-4)</code>
Table 10	<code>inverse_design_probe.py --target ... --objective ...</code>
Section 2.2	<code>cross.py</code>
Table 4	<code>tierC.py</code> on jed, SLURM job 66552359 (12 min, 96 GB)
Table 3	<code>plane_integrals.py</code>
Table 9	<code>active_subspace.py --target {full3d,interface}</code>
Table 11	<code>learning_curves.py --target ...</code>

The composed pipeline. The velocity override is `alumin_velocity_from_ascii` in `alumin/load_data.mac`, the metric export `alumin_write_midacd` in `alumin/post_processing.mac`, both off by default. Predictions are written in alucell’s table format by `surrogate_velocity_for_alumin.py --run <id>`. Driving the alumina model needs a stationary database, which the campaign deletes (`stat_dataset/clean_dbf`). One run was re-solved with a private copy of the macro tree in which that deletion is removed. Note that the deletion has to be gated inside that macro and not at its call site — everything above it is the write-out, so skipping the macro leaves a database holding the mesh and no solution.

Cost and identifiability. The unit price is `sbatch --test-only` on an eight-task, eight-hour job. The rank and conditioning of the pooled design, and the trade-off table of Section 8.8, are from `design_identifiability.py`, which builds the actual design matrix — Paley–Hadamard foldover rows, base runs, interior points, each geometry’s row of the geometry basis — and takes its singular values.

The geometry pilot. Meshes, solves and labelling dumps are on `jed.hpc.epfl.ch` under `/work/gr-pi/alu-data/CNG/CNG2/geom_pilot/`, as `g01–g12` (meshes), `s_g01–s_g12` (solves, each a `run.h5`) and `d_g01–d_g12` (the undeformed-mesh dumps). The design point of each is in its own `data/_geometrical_params.mac`. Relabelling and the spectra: `canonical_relabel.py {cuveb,inte}`. All model-touching scripts require `/home/barucca/miniconda3/bin/python` (torch 2.6.0+cu124). The campaign master is `/work/gr-pi/alu-data/CNG/CNG2/runs_two/` on `jed.hpc.epfl.ch`; `master_dataset.h5` there is complete at 188 658 553 168 bytes, the local copy being an interrupted transfer short by 4 689 015 120 bytes.

References

- [1] L. Sirovich, “Turbulence and the dynamics of coherent structures, Part I: coherent structures”, *Quarterly of Applied Mathematics*, 1987. — the method of snapshots used throughout.
- [2] G. Berkooz, P. Holmes, J. L. Lumley, “The proper orthogonal decomposition in the analysis of turbulent flows”, *Annual Review of Fluid Mechanics*, 1993.

- [3] P. Benner, S. Gugercin, K. Willcox, “A survey of projection-based model reduction methods for parametric dynamical systems”, *SIAM Review*, 2015. — the parametric setting of Section 8.
- [4] P. C. Hansen, *Rank-Deficient and Discrete Ill-Posed Problems: Numerical Aspects of Linear Inversion*, SIAM, 1998. — truncated SVD, filter factors, and the reading of a singular spectrum.
- [5] H. W. Engl, M. Hanke, A. Neubauer, *Regularization of Inverse Problems*, Kluwer, 1996.
- [6] P. G. Constantine, *Active Subspaces: Emerging Ideas for Dimension Reduction in Parameter Studies*, SIAM Spotlights, 2015.
- [7] P. G. Constantine, E. Dow, Q. Wang, “Active subspace methods in theory and practice: applications to kriging surfaces”, *SIAM Journal on Scientific Computing*, 2014.
- [8] A. Jameson, “Aerodynamic design via control theory”, *Journal of Scientific Computing*, 1988.
- [9] M. B. Giles, N. A. Pierce, “An introduction to the adjoint approach to design”, *Flow, Turbulence and Combustion*, 2000.
- [10] G. E. P. Box, J. S. Hunter, W. G. Hunter, *Statistics for Experimenters*, 2nd ed., Wiley, 2005. — foldover designs and aliasing.
- [11] A. S. Hedayat, N. J. A. Sloane, J. Stufken, *Orthogonal Arrays: Theory and Applications*, Springer, 1999. — Hadamard and Paley constructions.
- [12] A. C. Atkinson, A. N. Donev, R. D. Tobias, *Optimum Experimental Design, with SAS*, Oxford University Press, 2007.
- [13] A. Saltelli et al., *Global Sensitivity Analysis: The Primer*, Wiley, 2008. — variance-based sensitivity, contrasted with the directional question asked here.
- [14] T. J. R. Hughes, L. P. Franca, M. Balestra, “A new finite element formulation for computational fluid dynamics: V. Circumventing the Babuška–Brezzi condition”, *Computer Methods in Applied Mechanics and Engineering*, 1986.
- [15] F. Brezzi, J. Pitkäranta, “On the stabilization of finite element approximations of the Stokes equations”, in *Efficient Solutions of Elliptic Systems*, Vieweg, 1984.
- [16] D. Boffi, F. Brezzi, M. Fortin, *Mixed Finite Element Methods and Applications*, Springer, 2013. — the Piola transform and $H(\text{div})$ -conforming spaces.
- [17] P. E. Farrell, M. D. Piggott, C. C. Pain, G. J. Gorman, C. R. Wilson, “Conservative interpolation between unstructured meshes via supermesh construction”, *Computer Methods in Applied Mechanics and Engineering*, 2009.
- [18] K. Patouillet, N. Chailly, B. Allano, A. Clark, J. Perry, M. Vázquez, “Optimization of alumina feeding in electrolysis cells using multi-physics modeling and deep learning surrogate”, in *Light Metals 2025*, TMS, 2025. — the downstream half of the chain of Section 10, surrogated with the same solver.
- [19] T. Hofer, *Numerical Simulation and Optimization of the Alumina Distribution in an Aluminium Electrolysis Pot*, PhD thesis no. 5023, École Polytechnique Fédérale de Lausanne, 2011. — the origin of the `alumin` module used here; objective in its eq. (8.3.4), load-weight results in its Table 10.4.
- [20] B. Moxnes, A. Solheim, M. Liane, E. Svinsaa, A. Halkjelsvik, “Improved cell operation by redistribution of the alumina feeding”, in *Light Metals 2009*, TMS, 2009.
- [21] A. van Haaren, H. Vogel, N. Hahn, R. Düssel, “Industry case study: optimisation of alumina feed cycle according to anode process operation”, in *Light Metals 2025*, TMS, 2025.
- [22] T. Sele, “Instabilities of the metal surface in electrolytic alumina reduction cells”, *Metallurgical Transactions B*, 1977.

- [23] V. Bojarevics, M. V. Romerio, “Long waves instability of liquid metal–electrolyte interface in aluminium electrolysis cells: a generalization of Sele’s criterion”, *European Journal of Mechanics B/Fluids*, 1994.

Author, title, venue and year above are given as used; volume and page numbers should be checked against the publisher before submission.